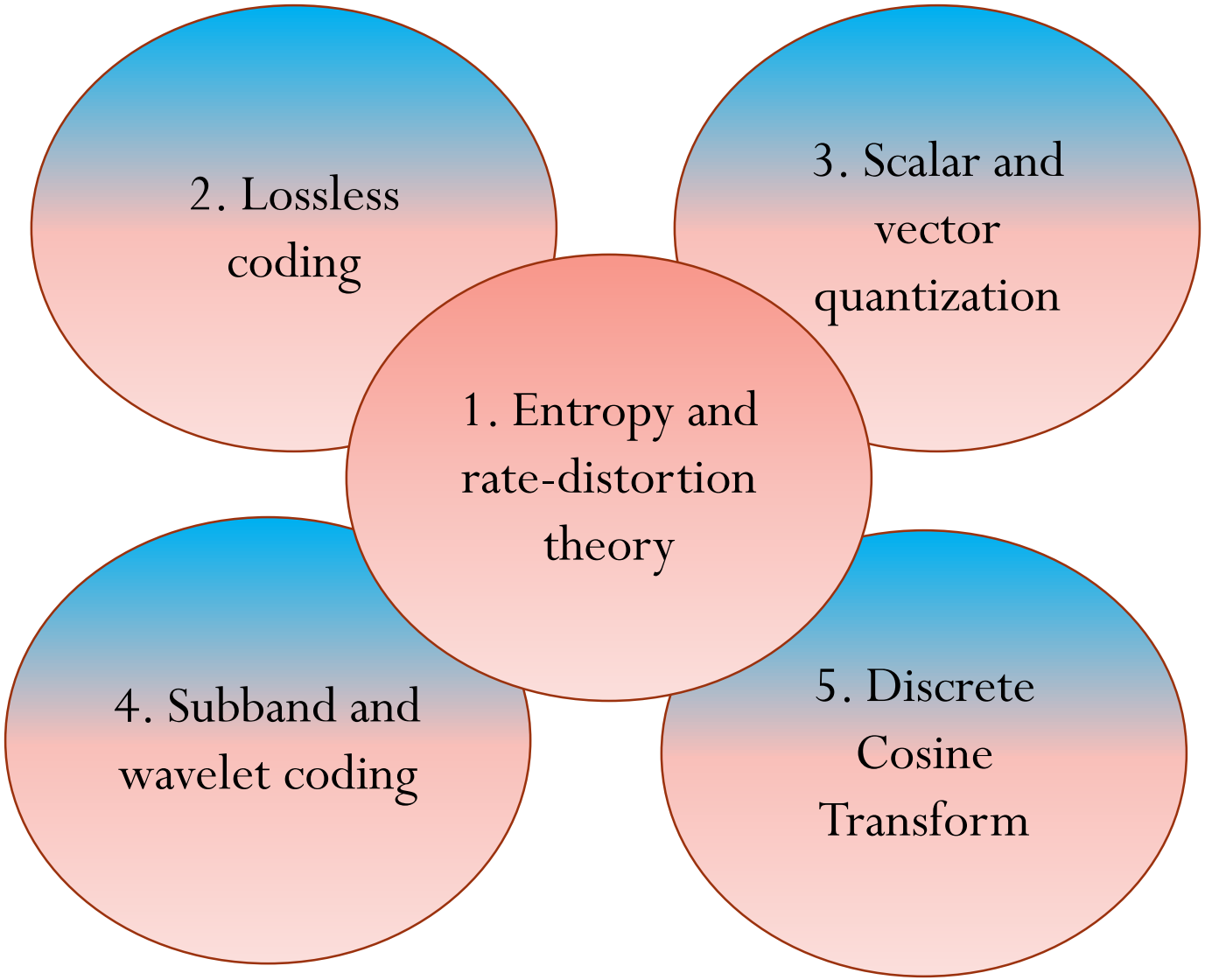
Chapter 7 Image Compression

Jing-Ming Guo (郭景明)

886-02-27303241
E-mail: jmguo@mail.ntust.edu.tw
Advanced Intelligent Image and Vision Technology Research Center
Dept. Electrical Engineering
National Taiwan University of Science and Technology
https://sites.google.com/view/ntust-mspl/jing-ming-guo

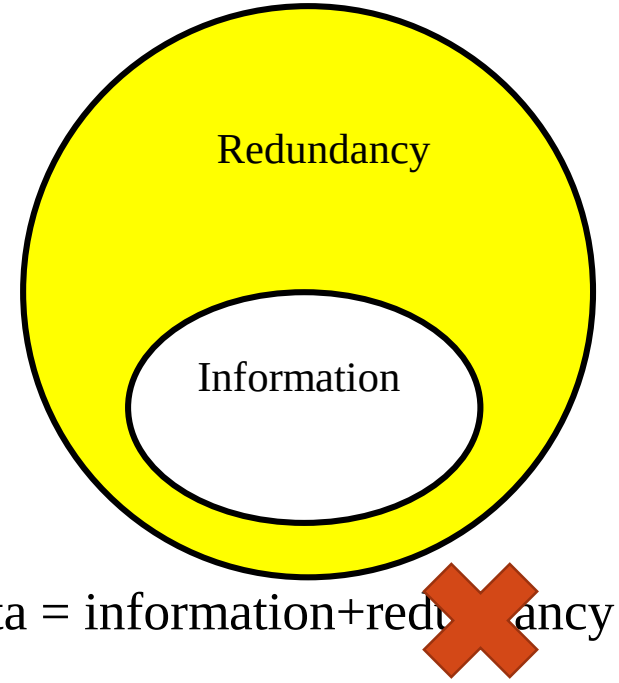
Outline


2. Lossless coding
3. Scalar and vector quantization
1. Entropy and rate-distortion theory
4. Subband and wavelet coding
5. Discrete Cosine Transform

Entropy and Rate-Distortion Theory

Why Do We Need Compression?

- Requirements may outstrip the anticipated increase of storage space and bandwidth
- For data storage and data transmission
  - DVD
  - Video conference
  - Printer



Data = information+redu ancy

1920x1080x3(RGB)x1(Byte)x60(fps)

=373 MB data generated in one second.➔ need compression!!

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Image Quality Measurement

Objective measurement

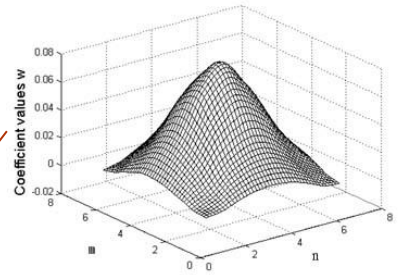
- Mean Square Error (MSE)

$$MSE = {\sigma_e}^2 = \frac{1}{N}\sum_i\sum_j (x_{i,j} - {x'}_{i,j})^2$$

$x_{i,j}$: Original image
${x'}_{i,j}$: Reconstructed image
N : Overall pixel number

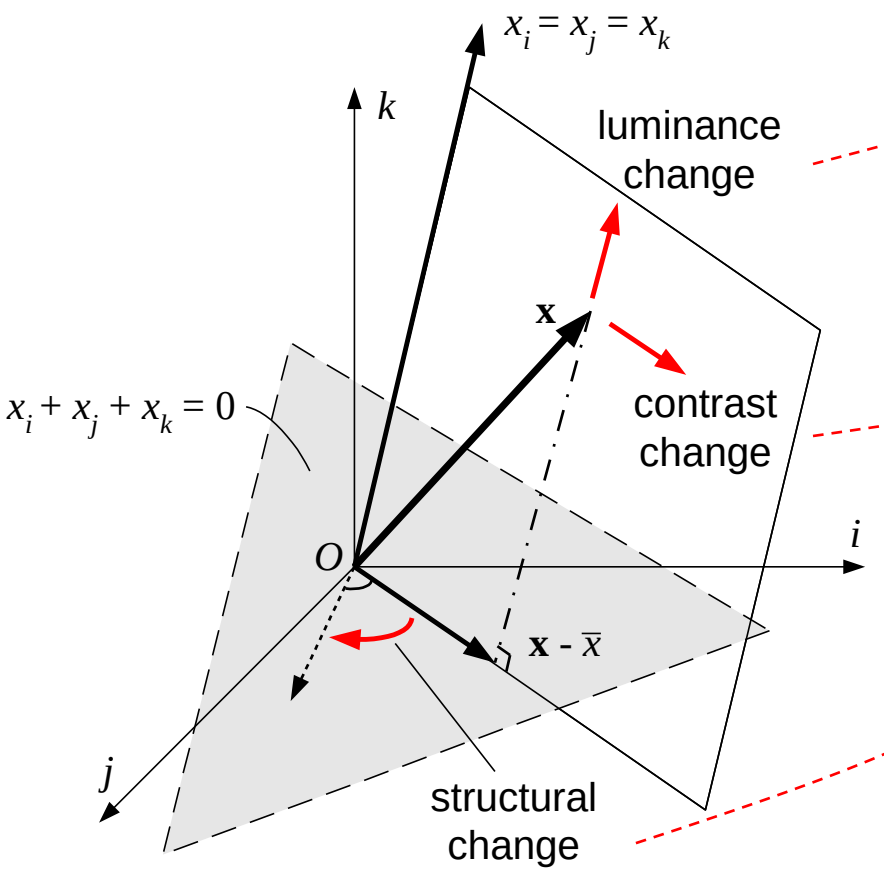
- Peak Signal to Noise Ratio (PSNR)
  - $$PSNR = 10\log_{10}\frac{x_{max}^2}{\sigma_e^2} \qquad (x_{max} = 255)$$
  - 40↑: excellent, 30~40: good, 20~30: poor, 20↓: unacceptable
- Human visual Peak Signal to Noise Ratio (HPSNR)

$$HPSNR = 10\log_{10}\left(\frac{{x_{max}}^2}{\frac{1}{N}\sum_i\sum_j[\sum_m\sum_n w_{m,n}\left(x_{i+m,j+n} - {x'}_{i+m,j+n}\right)]^2}\right)$$



Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Structural Similarity Index (SSIM)



$$l(x,y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}$$

$$c(x,y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}$$

$$s(x,y) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3}$$

$$where\ \sigma_{xy} = \sum_x \sum_y (x - \mu_x)(y - \mu_y)\ p(x,y)$$

$$SSIM(x,y) = l(x,y) \cdot c(x,y) \cdot s(x,y)$$

$$\mu_x = \sum_i w_i x_i, \quad \sigma_x^2 = \sum_i w_i (x_i - \mu_x)^2, \quad \sigma_{xy} = \sum_i w_i (x_i - \mu_x)(y_i - \mu_y) p(x,y)$$

In practice

[Wang & Bovik, *IEEE Signal Processing Letters*, '02]

[Wang *et al.*, *IEEE Trans. Image Processing*, '04]

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Information and Entropy

- **Meaning of information:**

  A. The sun rises from East

  B. A level 10 earthquake occurred yesterday

  ★The information is inversely proportional to the probability of the event.

$$I(x) = \log_a \frac{1}{p(x_j)} = -\log_a p\,(x_j)$$

  ★Example: A uniform image pattern is with 16 colors. what is the information of each color?

$$I(x) = -\log_2 \frac{1}{16} = 4\ bits$$

- **Meaning of entropy:** Average information of a set of events

The best that a lossless compression scheme can do is to encode the output of a source with an average number of bits equal to the entropy of the source.

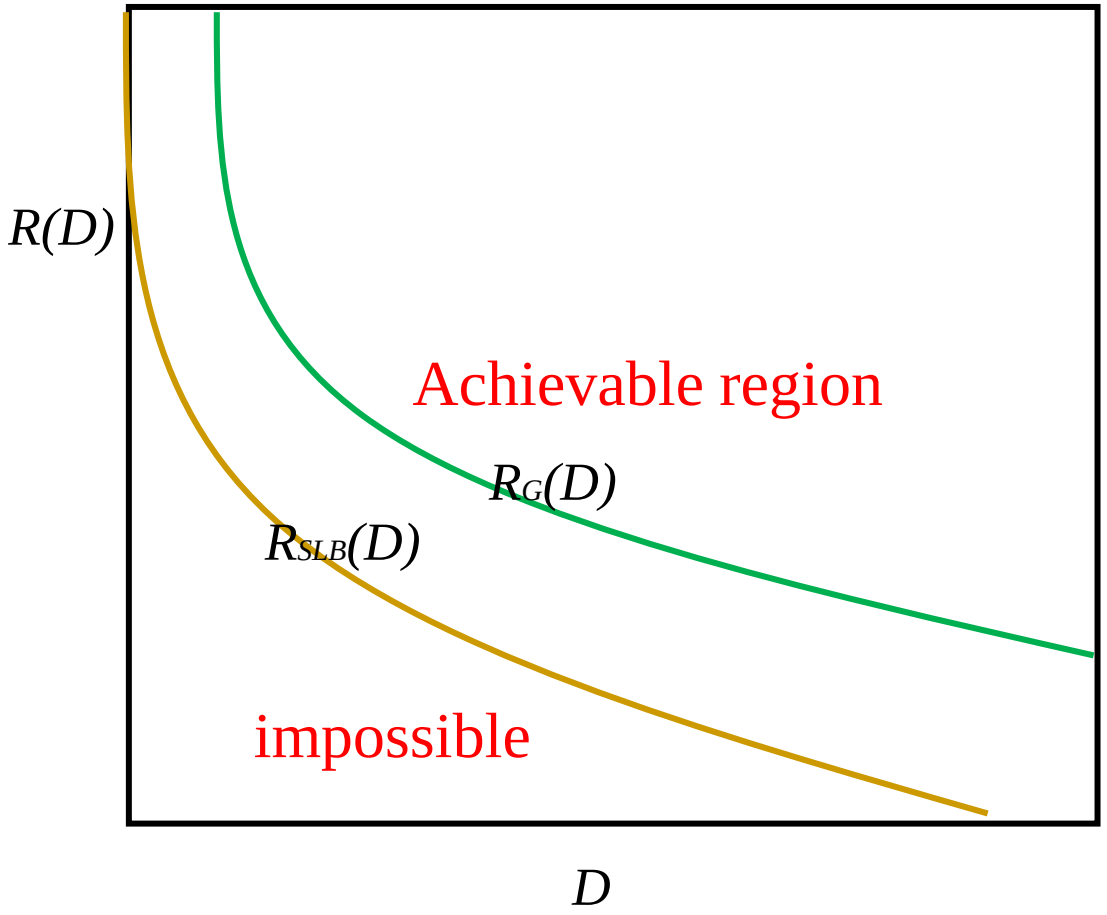
$$H(x) = \sum_{i=1}^{N} p(x_i) I(x_i) = -\sum_{i=1}^{N} p(x_i) \log_2 p\,(x_i)$$

Proven by Shannon

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Rate distortion theory

- Rate distortion theory is concerned with the trade-offs between distortion and rate in lossy compression schemes.



R(D) for Gaussian can be viewed as an upper bound for the rate distortion for any input.

$$R_{SLB}(D) = h(x) - \frac{1}{2}\log(2\pi eD)$$

where

$$h(x) = \int f_x(x)\log(\frac{1}{f_x(x)})dx$$

Derivations are available in [4] and [5].

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Lossless Coding

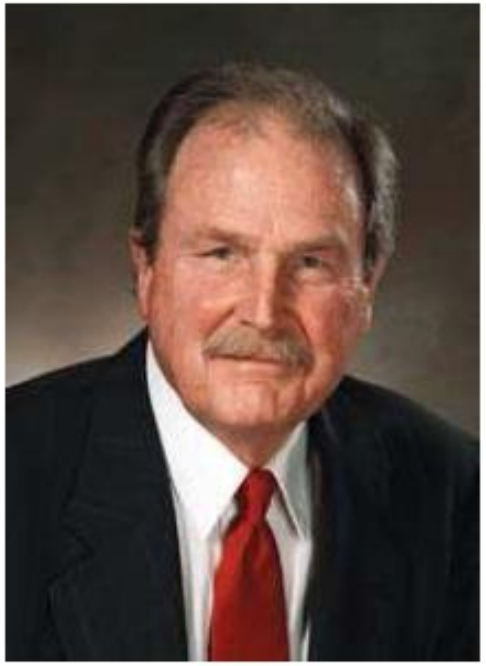
Lossless Coding Technologies

- Run-length coding
- Shannon Feno coding: Proposed by Claude E. Shannon and Robert M. Fano in 1960
- LZW: Proposed by Abranham Lempel, Jacob Ziv, and Terry Welch in 1984
- Huffman coding: Proposed by David A. Huffman in 1952
- Arithmetic coding: Proposed by IBM (Jorma Rissanen)in 1976, patented in 1981 and due in 2001

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Huffman Coding

- A procedure to construct optimal prefix code
- Result of David A. Huffman's term paper in 1952 when he was a PhD student at MIT



(David A. Huffman, 1925-1999)

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Huffman Code Design

- Requirement:
  - The source probability distribution.

  (Not available in many cases)

- Procedure:

  1. Sort the probability of all source symbols in a descending order.

  2. Merge the last two into a new symbol, add their probabilities.

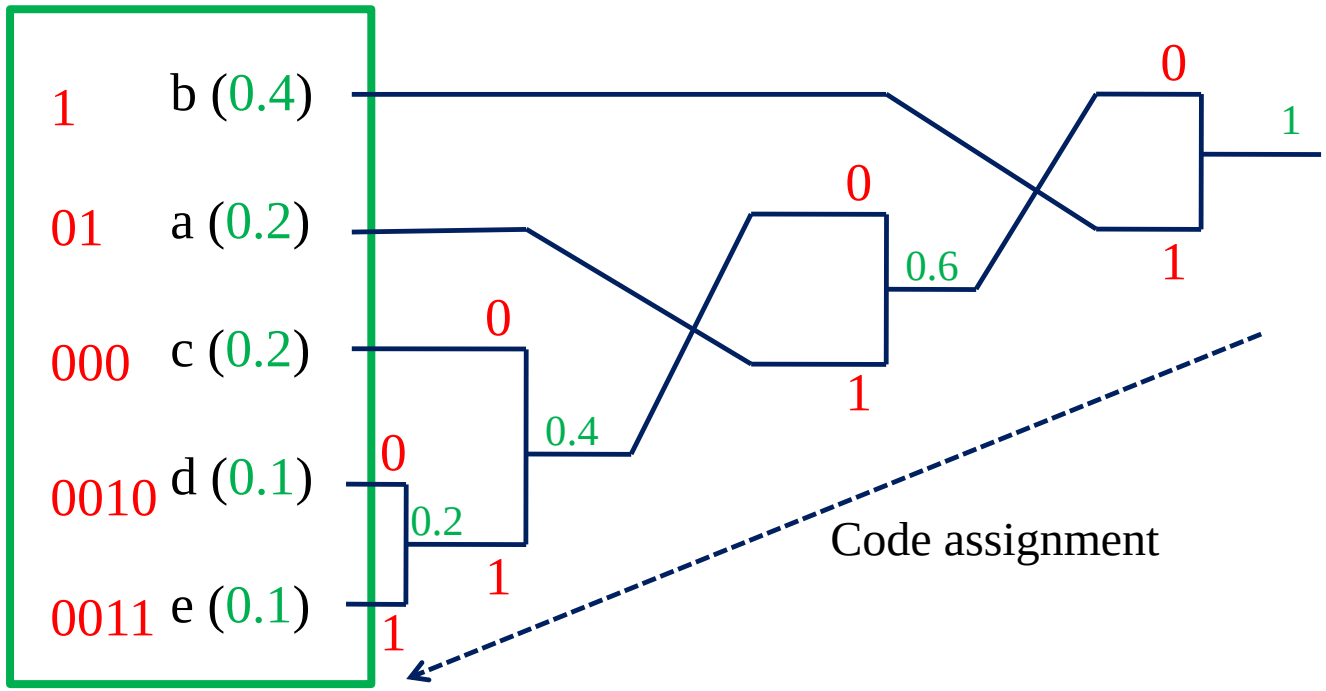
  3. Repeat Steps 1 and 2 until only one symbol (the root) is left.

  4. Code assignment: Traverse the tree from the root to each leaf node, assign 0 to the top branch and 1 to the bottom branch.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Example:

- Source alphabet A = {a, b, c, d, e}
- Probability distribution: {0.2, 0.4, 0.2, 0.1, 0.1}



Huffman code is prefix-free

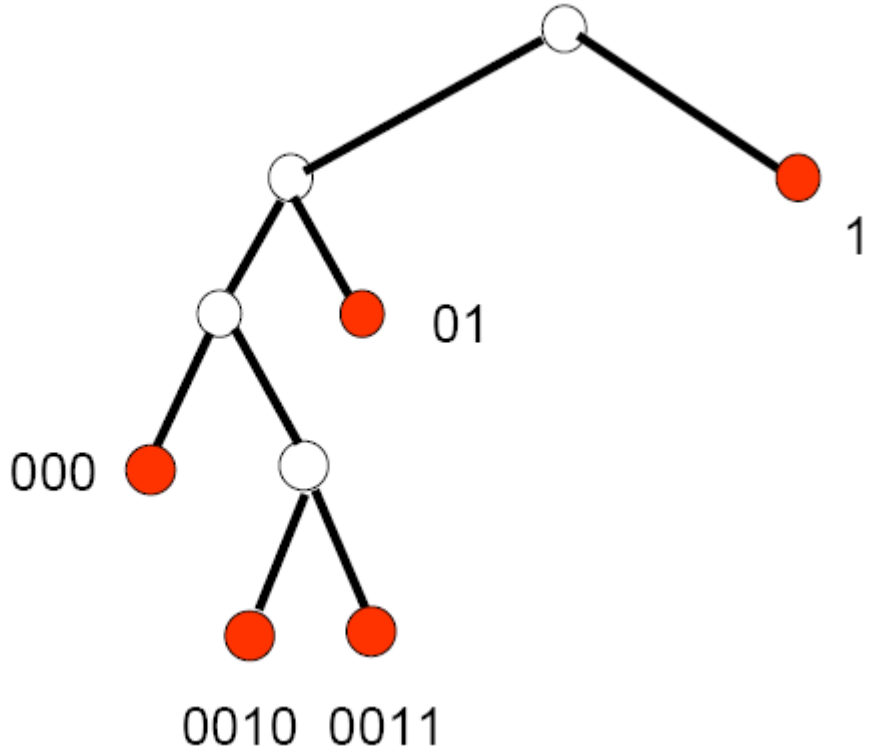
1

01

000

0010

0011



All codewords are leaf nodes

=>No code is a prefix of any other codes. (Prefix free)

Average Codeword Length vs Entropy

- Source alphabet A = {a, b, c, d, e}
- Probability distribution: {0.2, 0.4, 0.2, 0.1, 0.1}
- Code: {01, 1, 000, 0010, 0011}
- Entropy:

$$H(S) = -(0.2 \times \log_2(0.2) \times 2 + 0.4 \times \log_2(0.4) + 0.1 \times \log_2(0.1) \times 2)$$
$$= 2.122\ bits/symbol$$

- Average Huffman codeword length:

L = 0.2*2+0.4*1+0.2*3+0.1*4+0.1*4

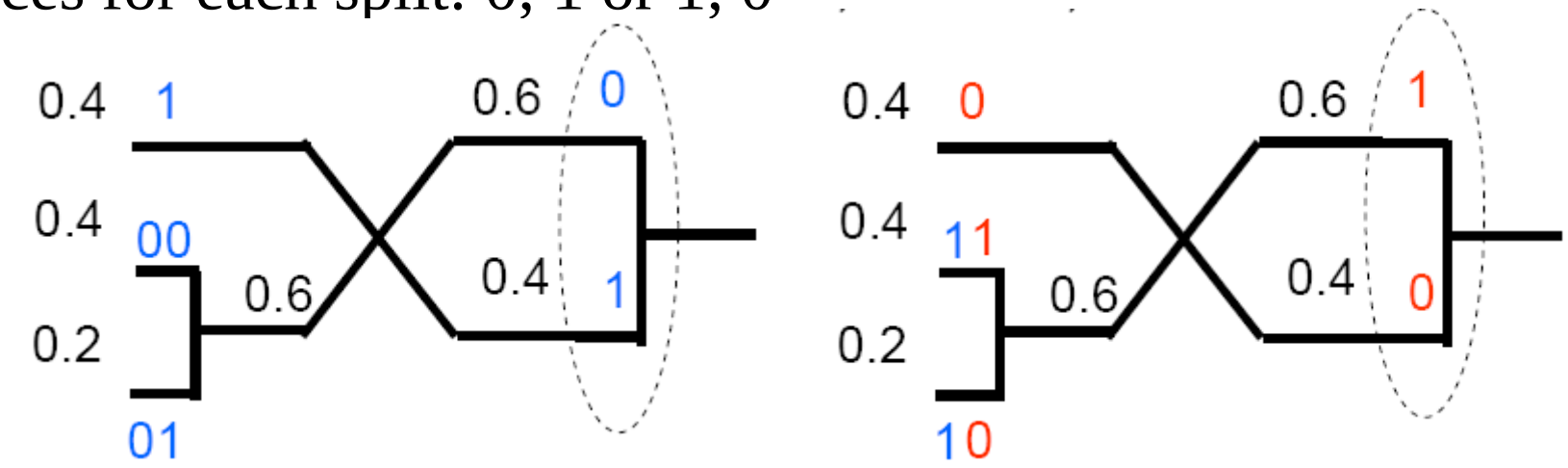
= 2.2 bits / symbol

- This verifies H(S) ≤ L < H(S) + 1.

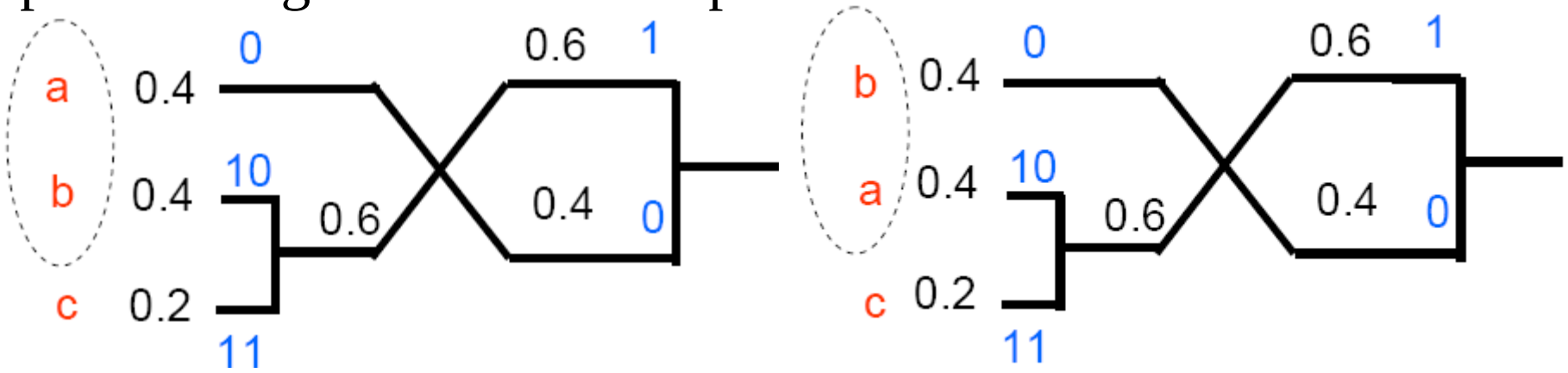
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Huffman Code is not unique

- Two choices for each split: 0, 1 or 1, 0



- Multiple ordering choices for tied probabilities



Extended Huffman Code

- Code multiple symbols jointly
  - Composite symbol: (X_1, X_2, …, X_k), and alphabet increased exponentially.

Example:

- P(Xj = 0) = P(Xj = 1) = ½
  - Entropy H(Xj) = 1 bit / symbol
- Huffman code for Xj: 0, 1

  Average code length 1 bit / symbol

Joint Prob. $P(X_{2i}, X_{2i+1})$

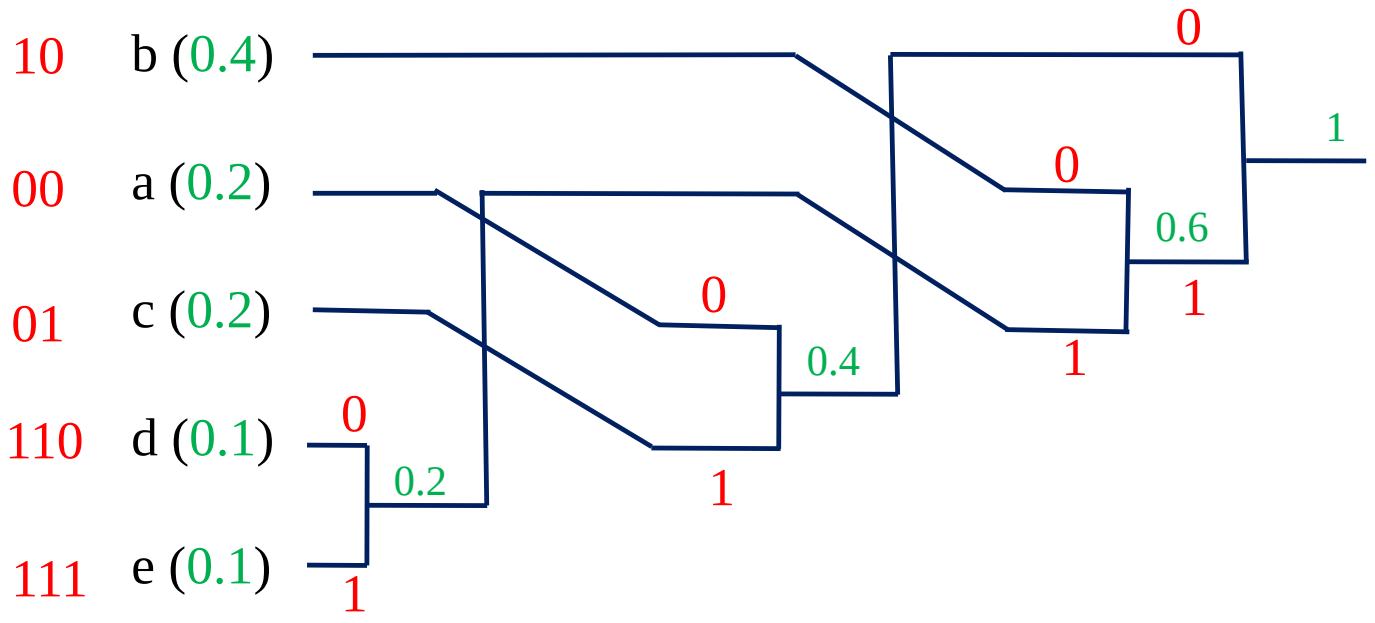
X_{2i} \ X_{2i+1}	0	1
0	3/8	1/8
1	1/8	3/8

- Joint probability: $P(X_{2i}, X_{2i+1})$
  - P(0,0) = 3/8, P(0,1) = 1/8, P(1,0) = 1/8, P(1,1) = 3/8
- Second order entropy: $\frac{1}{2}H(X_{2i}, X_{2i+1}) = 0.9056\ bits/symbol$
- Huffman code for (X_{2i}, X_{2i+1}): 00: 1, 11: 00, 01: 010, 10: 011
- Average code length: 0.9375 bits/symbol

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Minimum variance Huffman coding example:

- Source alphabet A = {a, b, c, d, e}
- Probability distribution: {0.2, 0.4, 0.2, 0.1, 0.1}



Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Minimum Variance Huffman Code

- Put the combined symbol as high as possible in the sorted list
- Repeat previous example:
  - Source alphabet A = {a, b, c, d, e}, probability distribution {0.2, 0.4, 0.2, 0.1, 0.1}
  - Minimum Variance Huffman codewords{00,10,01,110,111}
  - Compared to the previous code: {01, 1, 000, 0010, 0011}, average code length is identical 2.2 bits/sample
- Prevent unbalanced tree:
  - Reduce memory requirement for decoding
- Minimum Variance Huffman Code is good for hardware consideration.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Limitations of Huffman Code

- Need a probability distribution
  - Usually estimated from a training set
  - The practical data could be quite different
- Hard to adapt to changing statistics
  - Must design new codes on the fly
  - Context-adaptive method still need predefined tables
- Minimum codeword length is 1 bit
  - Serious penalty for high-probability symbols
  - Example: Binary source, P(0)=0.9

$Entropy: \ -0.9 \times \log_2(0.9) - 0.1 \times \log_2(0.1) = 0.469 \ bits$

  - Huffman code: 0, 1→Avg. code length: 1 bit
  - More than 100% redundancy !!!

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding

Huffman codes => integral number of bits long

Entropy of a symbol=> almost always a factional number

Theoretical possible compressed message is difficult achieved.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

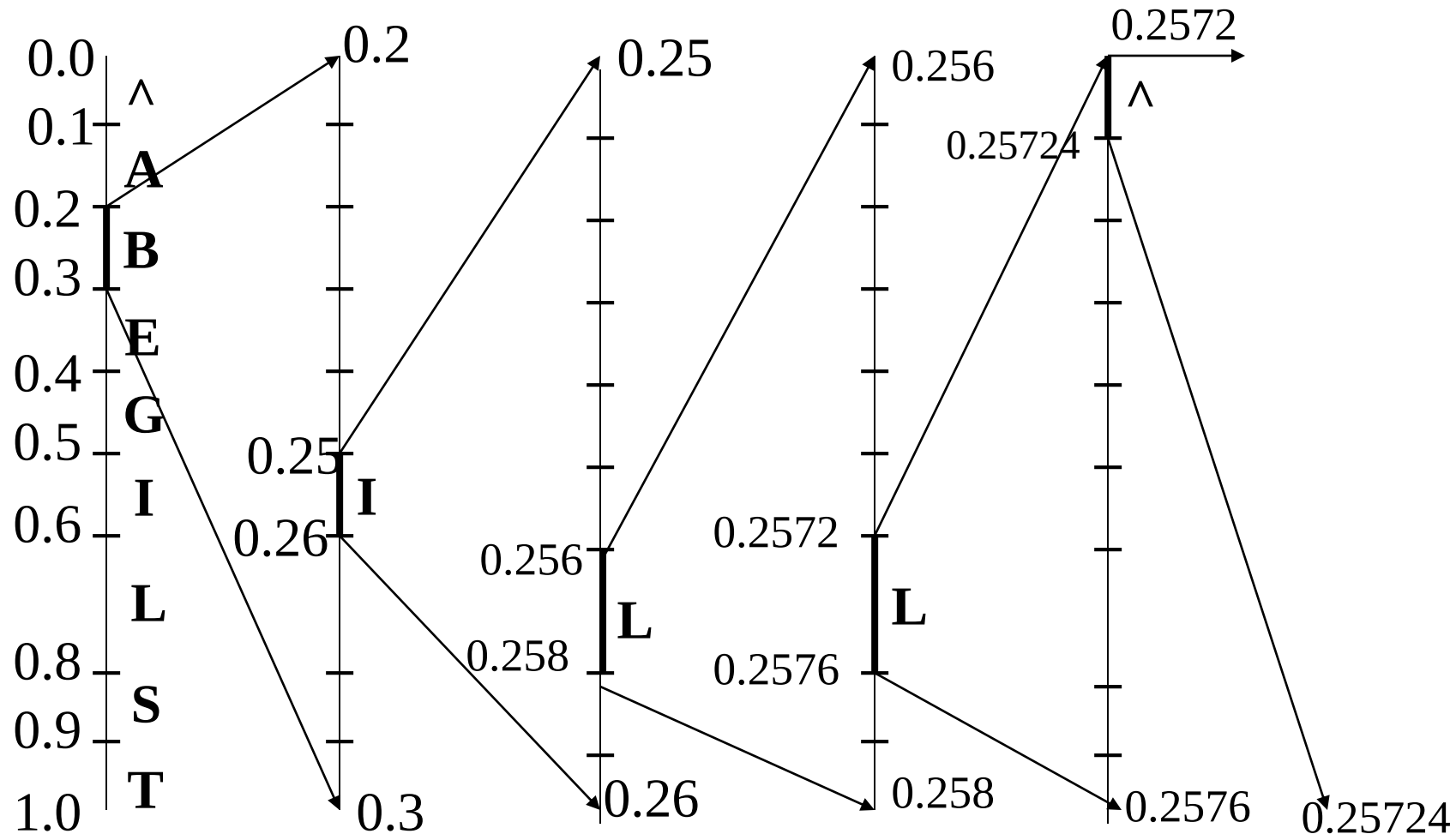
Arithmetic Coding Example (1)

Character	probability	Range
^(space)	1/10	$0.00 \le r < 0.10$
A	1/10	$0.10 \le r < 0.20$
B	1/10	$0.20 \le r < 0.30$
E	1/10	$0.30 \le r < 0.40$
G	1/10	$0.40 \le r < 0.50$
I	1/10	$0.50 \le r < 0.60$
L	2/10	$0.60 \le r < 0.80$
S	1/10	$0.80 \le r < 0.90$
T	1/10	$0.90 \le r < 1.00$

When the message BILL GATES is encoded.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding Example (1)



Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding Example (1)

New character	Low value	high value
B	0.2	0.3
I	0.25	0.26
L	0.256	0.258
L	0.2572	0.2576
^(space)	0.25720	0.25724
G	0.257216	0.257220
A	0.2572164	0.2572168
T	0.25721676	0.2572168
E	0.257216772	0.257216776
S	0.2572167752	0.2572167756

- The final value, named a ***tag***, 0.2572167752 will uniquely encode the message 'BILL GATES'.
- Any value between 0.2572167752 and 0.2572167756 can be a *tag* for the encoded message, and can be uniquely decoded.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding

- Encoding algorithm for arithmetic coding.

```
Low = 0.0 ; high =1.0 ;
while not EOF do
    range = high - low ; read(c) ;
    high = low + range×high_range(c) ;
    low = low + range×low_range(c) ;
enddo
output(low);
```


Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding

- Decoding is the inverse process.
- Since 0.2572167752 falls between 0.2 and 0.3, the first character must be ‘B’.
- Removing the effect of ‘B’ from 0.2572167752 by first subtracting the low value of B, 0.2, giving 0.0572167752.
- Then divided by the width of the range of ‘B’, 0.1. This gives a value of 0.572167752.
- Then calculate where that lands, which is in the range of the next letter, ‘I’.
- The process repeats until 0 or the known length of the message is reached.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

	r	c	Low	High	range
	0.2572167752	B	0.2	0.3	0.1
(0.2572167752-0.2)/0.1 =	0.572167752	I	0.5	0.6	0.1
(0.572167752 -0.5)/0.1 =	0.72167752	L	0.6	0.8	0.2
	0.6083876	L	0.6	0.8	0.2
.					
.	0.041938	^(space)	0.0	0.1	0.1
.					
.	0.41938	G	0.4	0.5	0.1
	0.1938	A	0.2	0.3	0.1
	0.938	T	0.9	1.0	0.1
	0.38	E	0.3	0.4	0.1
	0.8	S	0.8	0.9	0.1
	0.0				

Arithmetic Coding

- Decoding algorithm

```
r = input_code
repeat
    search c such that r falls in its range ;
    output(c) ;
    r = r - low_range(c) ;
    r = r/(high_range(c) - low_range(c));
until r equal 0
```


Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding Example (2)

Symbol	probability	Range
1	0.80	[0.00, 0.80)
2	0.02	[0.80, 0.82)
3	0.18	[0.82, 1.00)

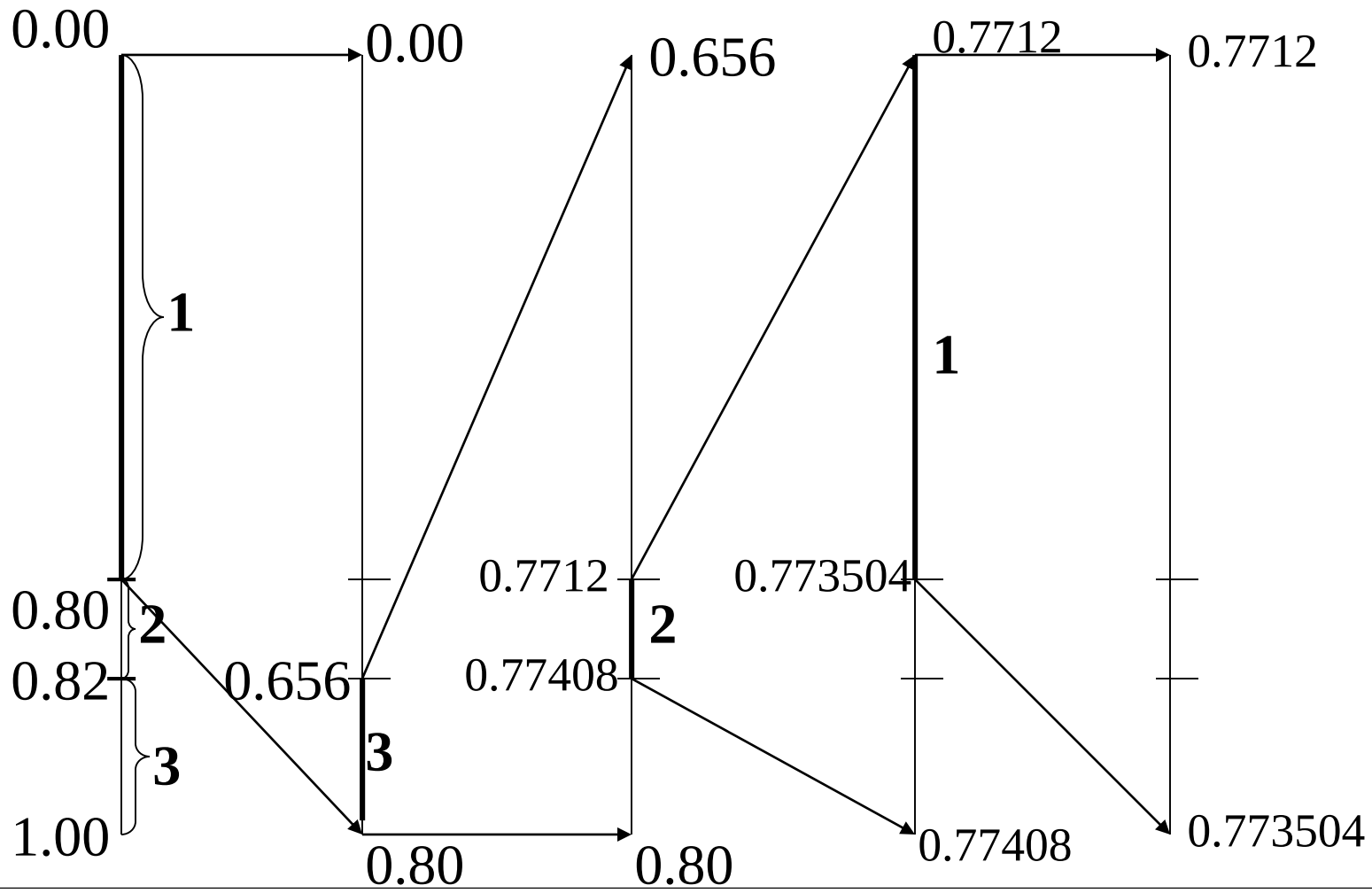
When the message **1 3 2 1** is encoded.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding Example (2)

Symbol	probability	Range
1	0.80	[0.00, 0.80)
2	0.02	[0.80, 0.82)
3	0.18	[0.82, 1.00)

When the message **1 3 2 1** is encoded.



Arithmetic Coding Example (2)

Encoding:

New character	Low value	High value
	0.0	1.0
1	0.0	0.8
3	0.656	0.800
2	0.7712	0.77408
1	0.7712	0.773504

$$\bar{T}_x(1312) = \frac{0.7712 + 0.773504}{2} = 0.772352$$

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Arithmetic Coding Example (2)

Decoding:

Symbol	probability	Range
1	0.80	[0.00, 0.80)
2	0.02	[0.80, 0.82)
3	0.18	[0.82, 1.00)

r	c	low	high	range	
0.772352	1	0	0.8	0.8	(0.772352-0)/0.8=0.96544
0.96544	3	0.82	1.0	0.18	(0.96544-0.82) / 0.18=0.808
0.808	2	0.8	0.82	0.02	(0.808-0.8)/0.02=0.4
0.4	1	0	0.8		

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Generating a Binary Code for Arithmetic Coding

- Problem:
  - The binary representation of some of the generated floating point values (tags) would be infinitely long.
  - We need increasing precision as the length of the sequence increases.
- Solution:
  - **Synchronized rescaling** and **incremental encoding**.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Higher-order and Adaptive Modeling

- To have a good compression ratio results in the statistical model compression methods, the model should be
  - Accurately predicts the frequency/ probability of symbols in the data stream.
  - A non-uniform distribution
- The finite context modeling provide a better prediction ability.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Higher-order and Adaptive Modeling

- Finite context modeling :
  - Calculate the probabilities for each incoming symbol based on the ***context*** (上下文) in which the symbol appears.
    - e.g. $p(u) = 0.05 \quad p(u|q) = 0.95$
  - The ***order*** of the model refers to the number of previous symbols that make up the context.
    - e.g. $p(x_n|x_{n-1}, x_{n-2}, \cdots, x_{n-k})$
  - In information theory, this type of finite context modeling is called Markov process/system.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Higher-order and Adaptive Modeling

- Problem:
  - As the order of the model increases linearly, the memory consumed by the model increases exponentially.
  - e.g. for q symbols and order k, the table size will be q^k.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Higher-order and Adaptive Modeling

- Adaptive modeling
  - In adaptive data compression, both the compressor and decompressor start with the same model.
  - The compressor encodes a symbol using the existing model, then it updates the model to account for the new symbol.
  - The decompressor likewise decodes a symbol using the existing model, then it updates the model.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Higher-order and Adaptive Modeling

- Adaptive data compression has a slight disadvantage in that it starts compressing with less than optimal statistics.
- By subtracting the cost of transmitting the statistics with the compressed data, however, an adaptive algorithm will usually perform better than a fixed statistical model.
- Adaptive compression also suffers in the cost of updating the model.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Higher-order and Adaptive Modeling

- Encoding phase

```
low = 0.0 ; high = 1.0 ;
while not EOF do
        read(c) ;
                range = high - low ;
                high = low + range *high_ range(context,c);
        low = low + range *low_ range(context,c);
                update_model(context,c);
            context = c ;
enddo
output(low);
```

Applications

The JBIG Standard

- JBIG --- Joint Bi-Level Image Processing Group
- JBIG was issued in 1993 by ISO/IEC for the progressive lossless compression of binary and low-precision gray-level images (typically, having less than 6 bits/pixel).
- The major advantages of JBIG over other existing standards are its capability of progressive encoding and its superior compression efficiency.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

The JBIG Standard

Context-based arithmetic coder

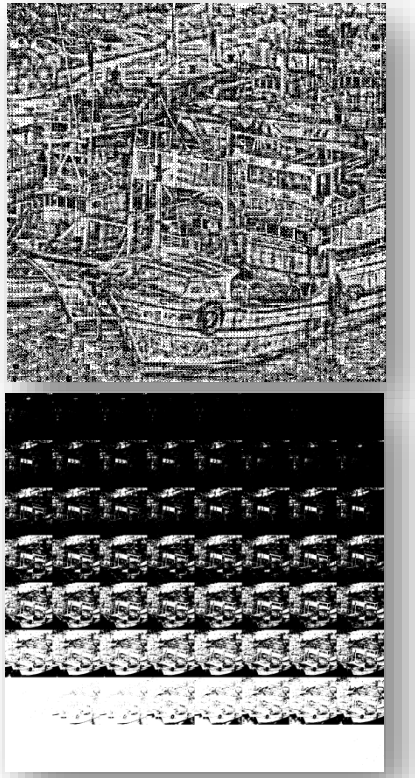
- The core of JBIG is an adaptive context-based arithmetic coder.
  - If the probability of encountering a black pixel p is 0.2 and the probability of encountering a white pixel q is 0.8.
  - Using a single arithmetic coder, the entropy is

$$H = -0.2\log_2 0.2 - 0.8\log_2 0.8 = 0.722$$

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

The JBIG Standard

Context-based arithmetic coder



- Group the data into Set A (80%) and Set B (20%), using two coders

$p_w = 0.95, p_b = 0.05, H_A = 0.286$

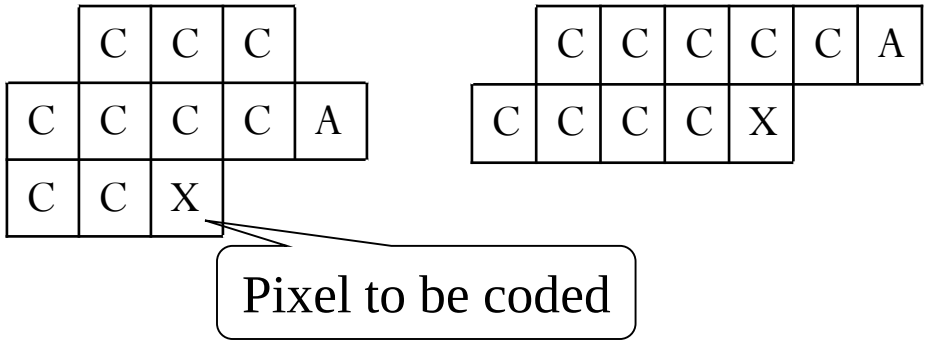
$p_w = 0.3, p_b = 0.7, H_B = 0.881$

then, the average $H = H_A *.8 + H_B *.2 = 0.405$.

A
B

P(w)=0.625
P(b)=0.375
H =0.954

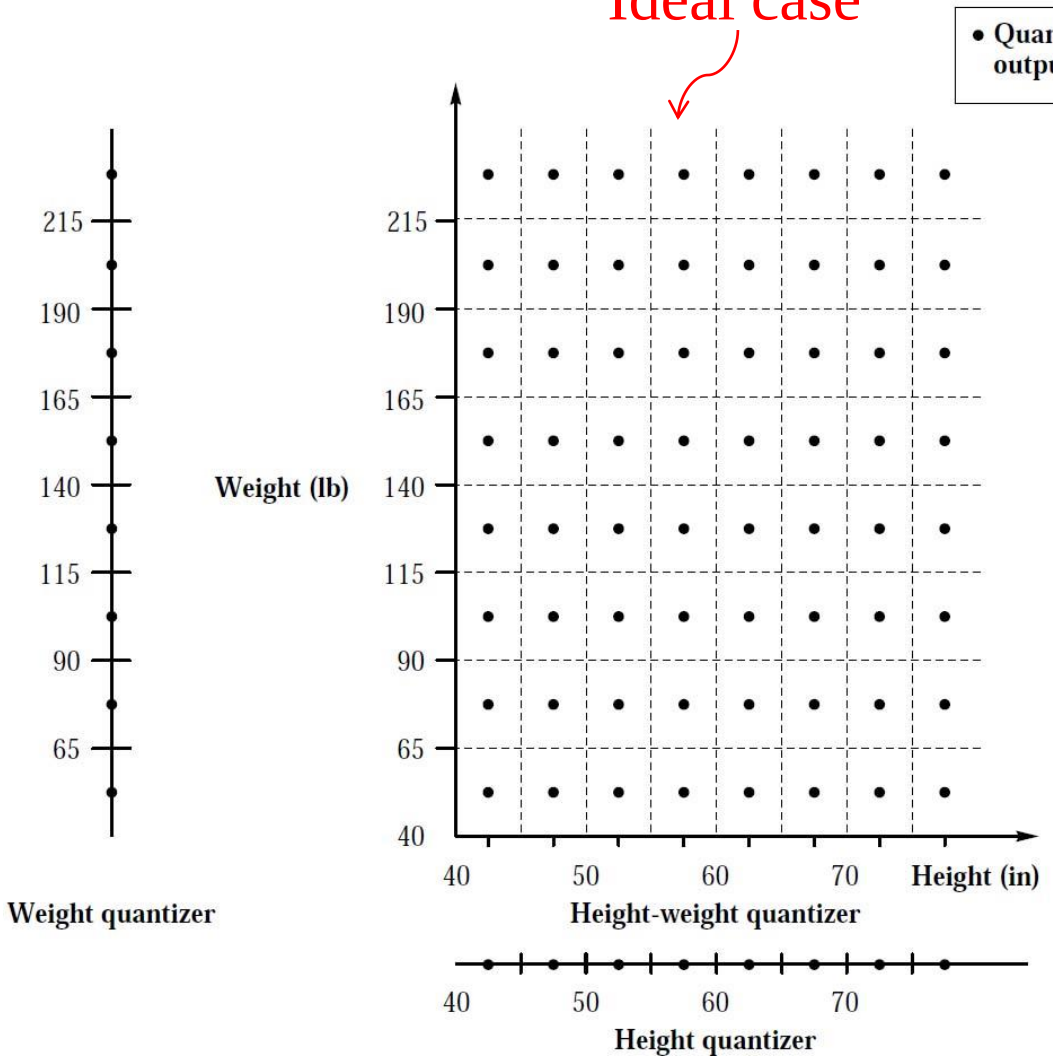
- The number of possible patterns is 1024. The JBIG coder uses 1024 or 4096 coders



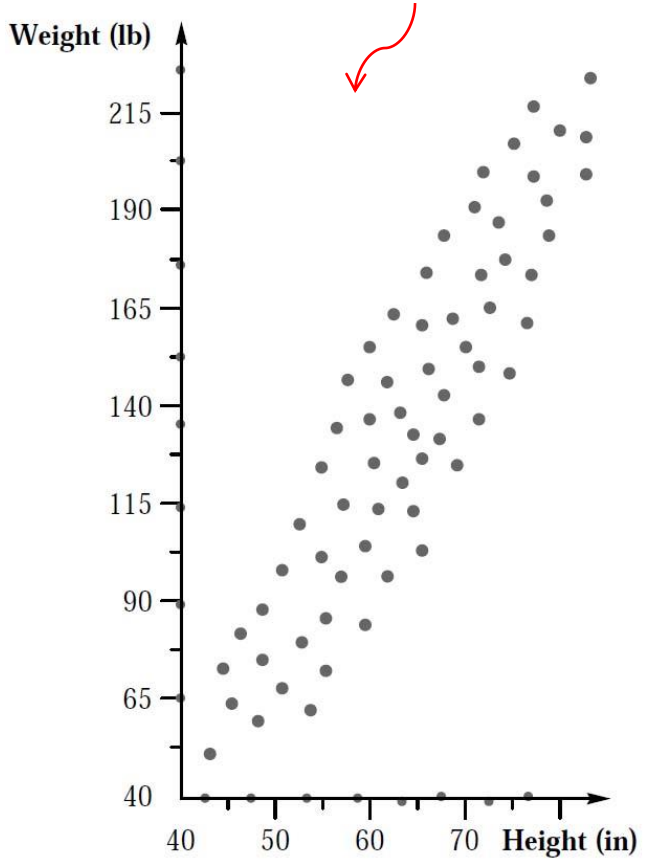
Scalar and Vector Quantization

Scalar quantization

Ideal case

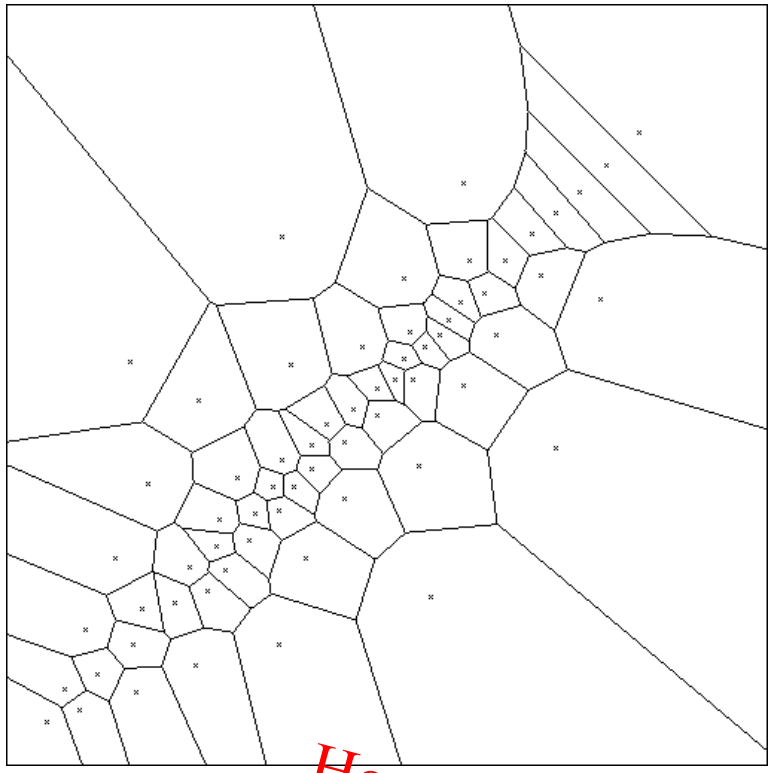


Practical case



Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Vector quantization



Codebook Generation

Linde-Buzo-Gray (LBG) algorithm

Let the codebook size be N_C and the training vectors be $\{ x(n) | n=1,\ldots, M\}$

Step 1 Let the initial codebook C=$\{ y(i) \mid i = 1,\ldots, N_C\}$ be randomly selected from $\{ x(n) | n = 1,\ldots, M\}$.

Step 2 Cluster the training vectors into N_C groups $G(i), i =1,\ldots,N_C$, where $G(i)=\{x(k) \mid d(x(k), y(i)) < d(x(k), y(j)),\ j \neq i$ and $d(p, q)$ denotes the distance between p and $q\}$

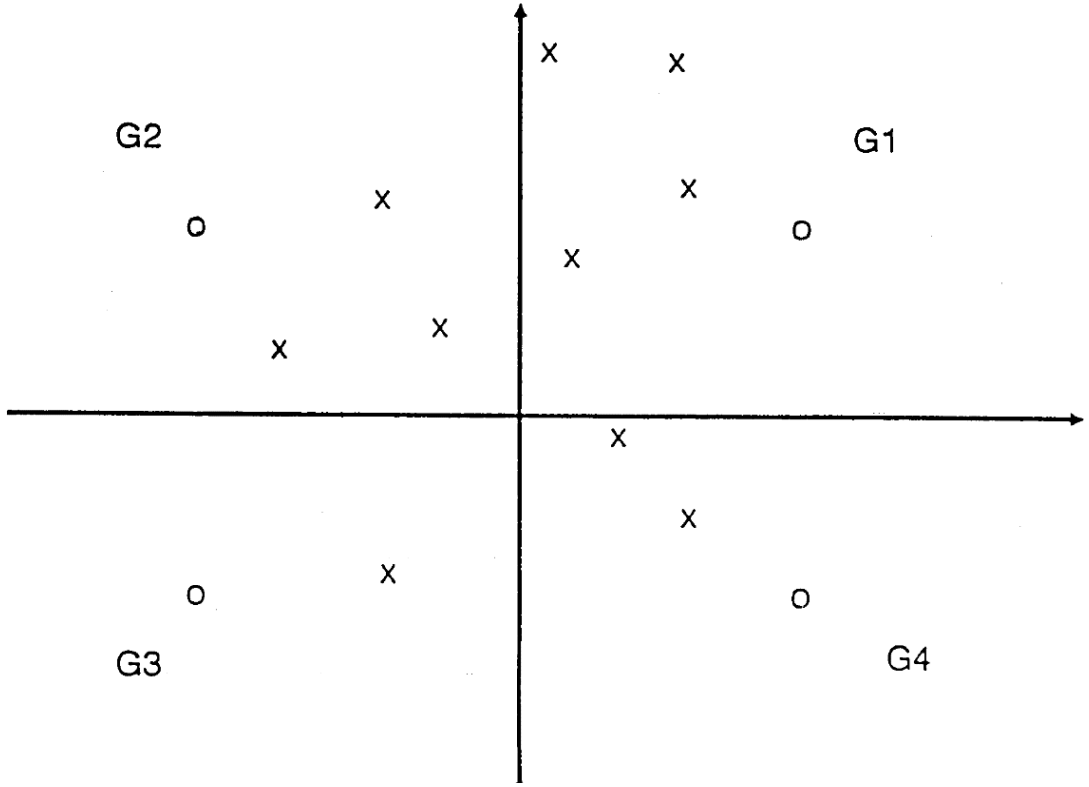
Step 3 New $y(i) = \frac{1}{|G(i)|} \sum_{x(k)\in G(i)} x(k)$, where |G(i)| = the number of vector in $G(i)$

Step 4 Compute the distortion $D = \sum_{i=1}^{N_C} \sum_{x(k)\in G(i)} d(x(k), y(i))$

Step 5 A threshold ε is selected, and the iteration continue until $\frac{D^{(l-1)} - D^{l}}{D^{(l-1)}} \leq \varepsilon$

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Codebook Generation

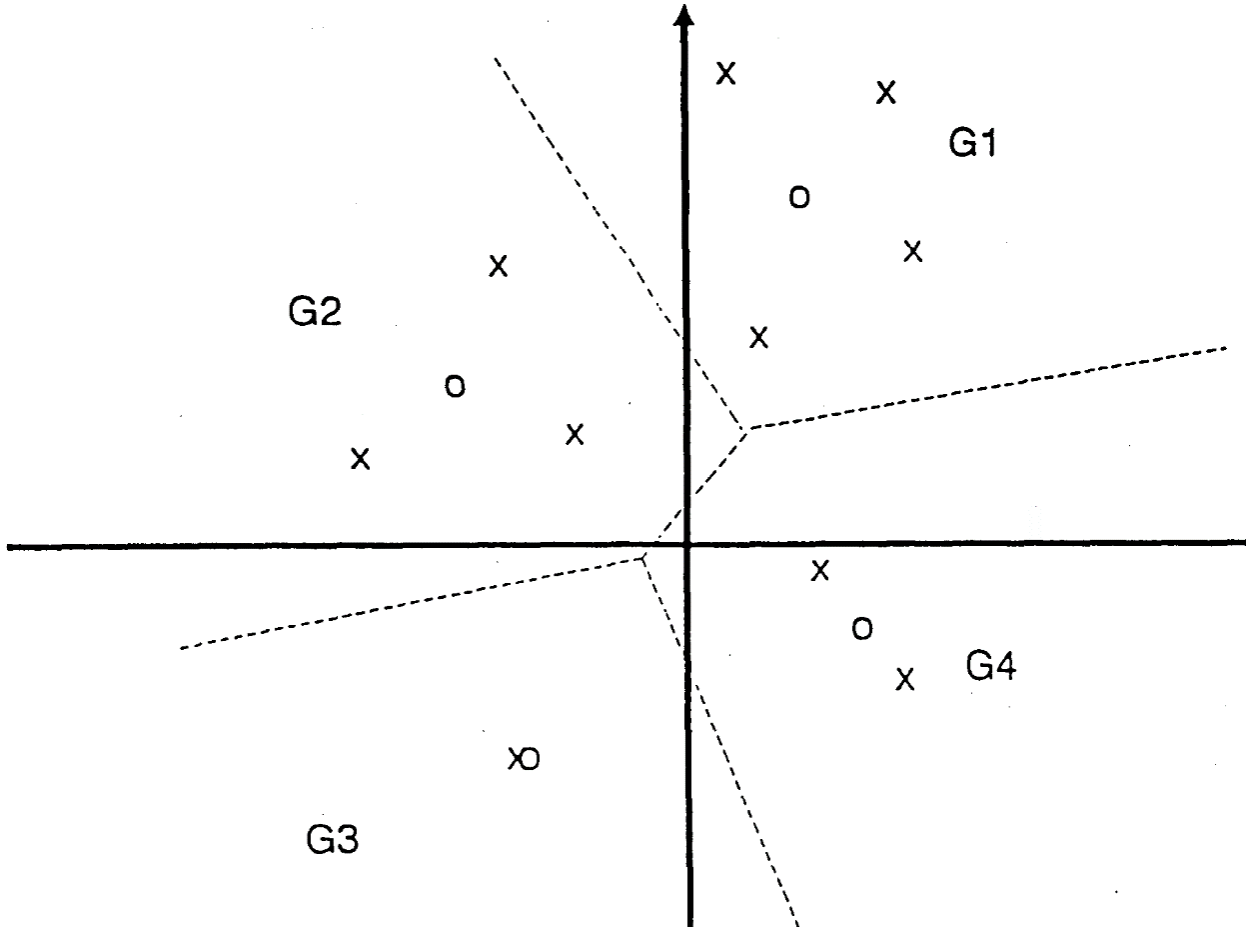


× = training codevectors

○ = codewords in the codebook

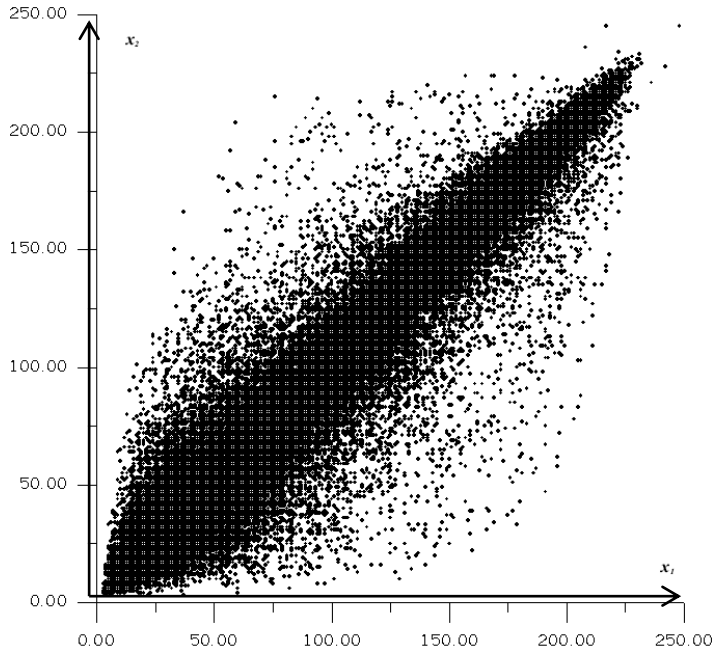
G*i* = region encoded into codeword *i*

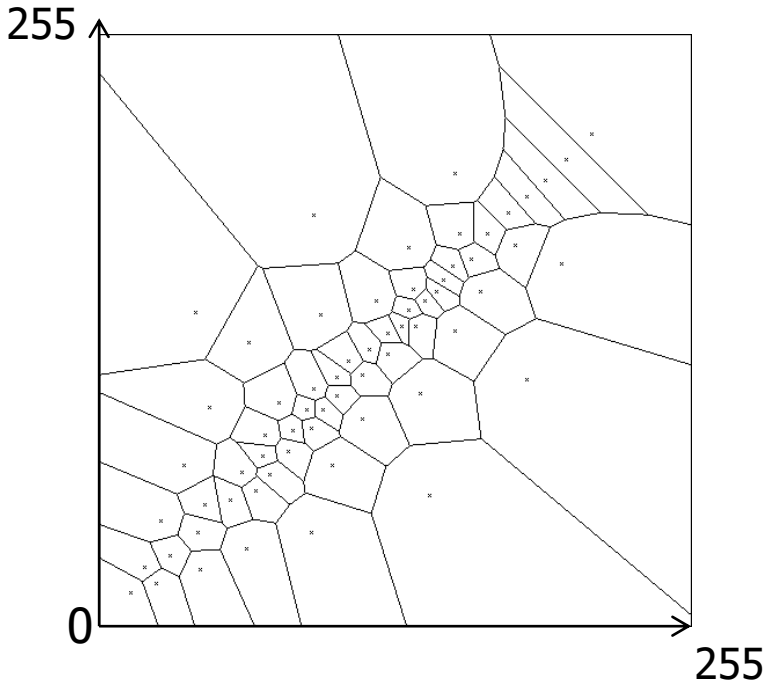
Codebook Generation



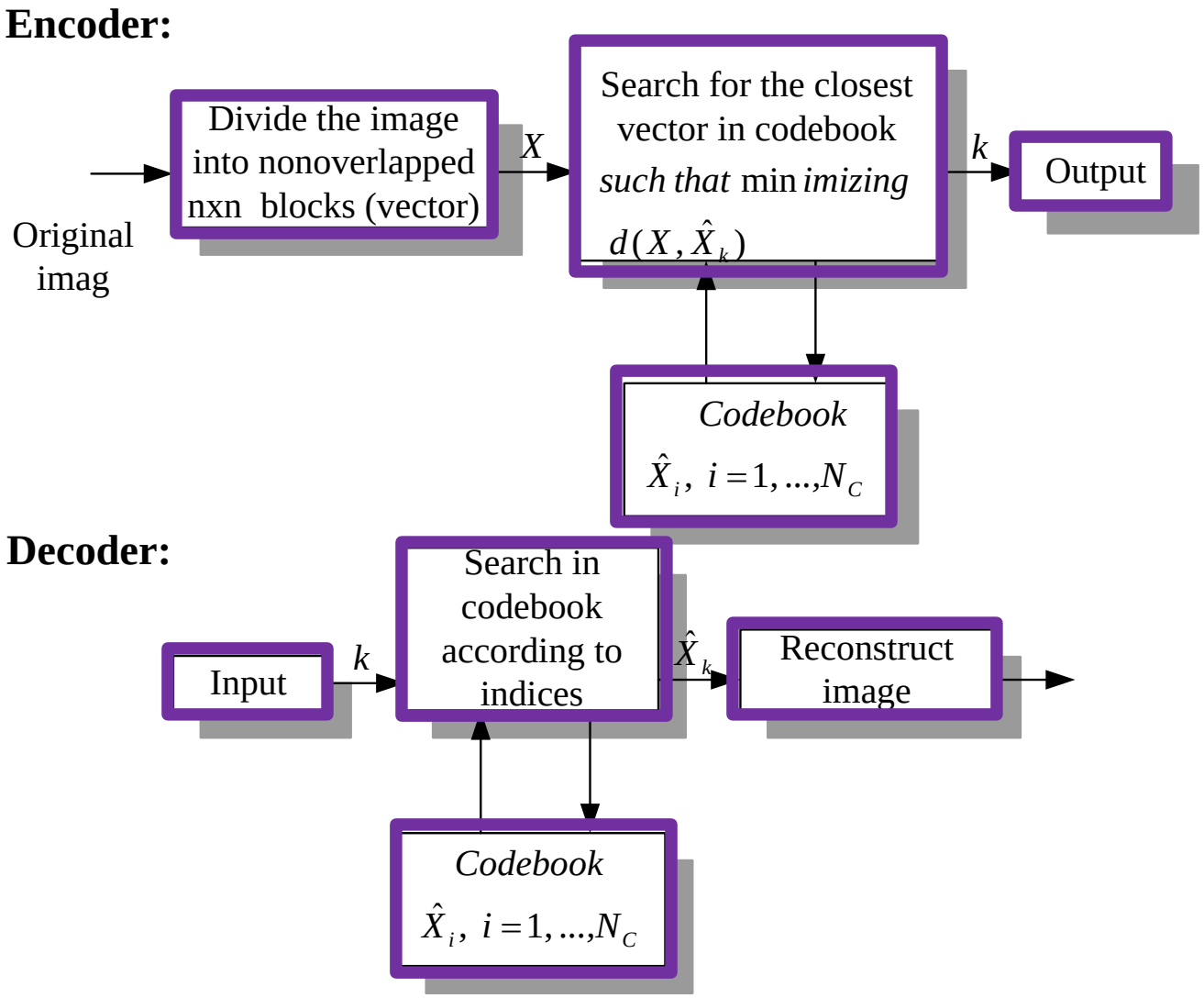
Codebook Generation

- An example of codebook using Lena
  - Block size is of 1x2
  - Nc = 64



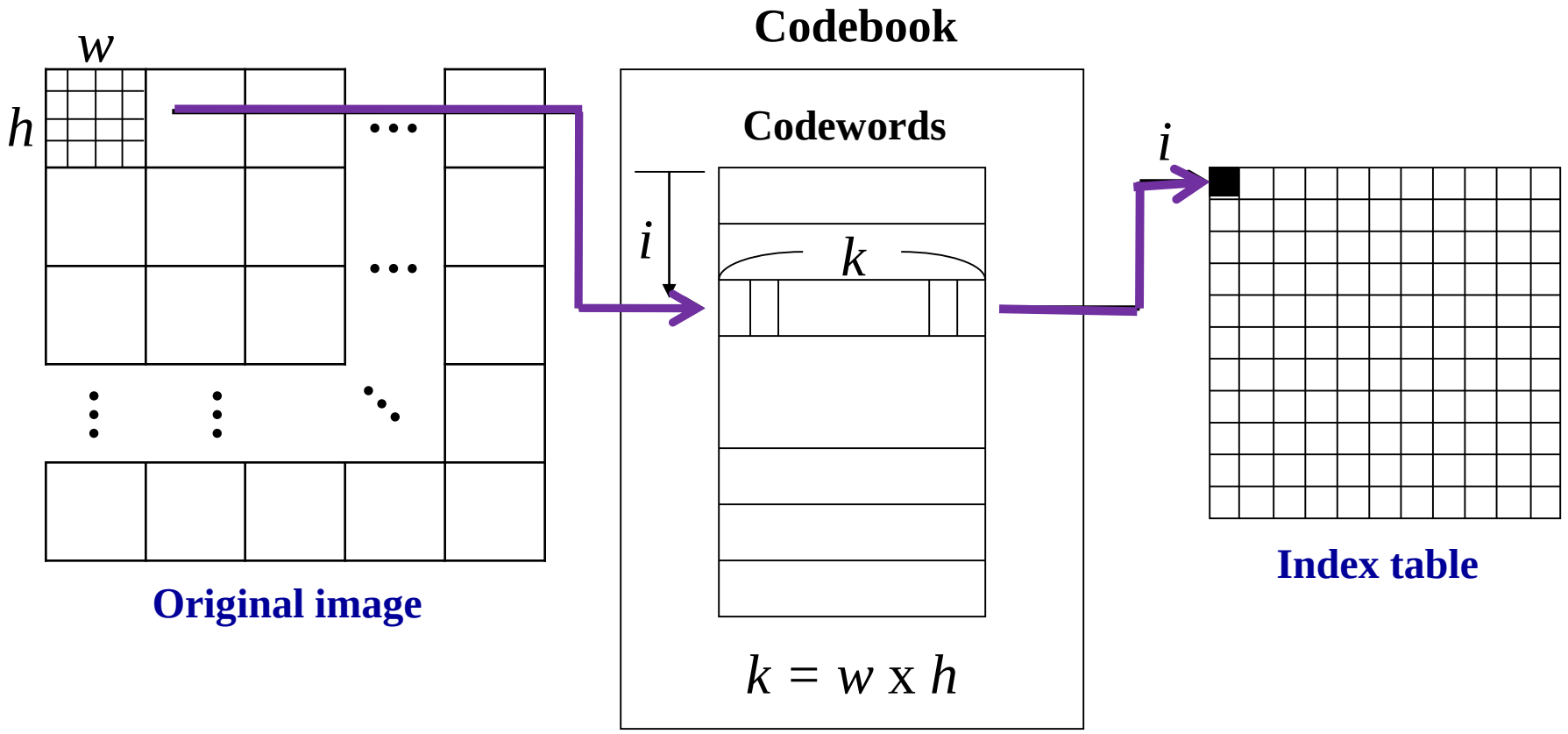


Introduction to VQ



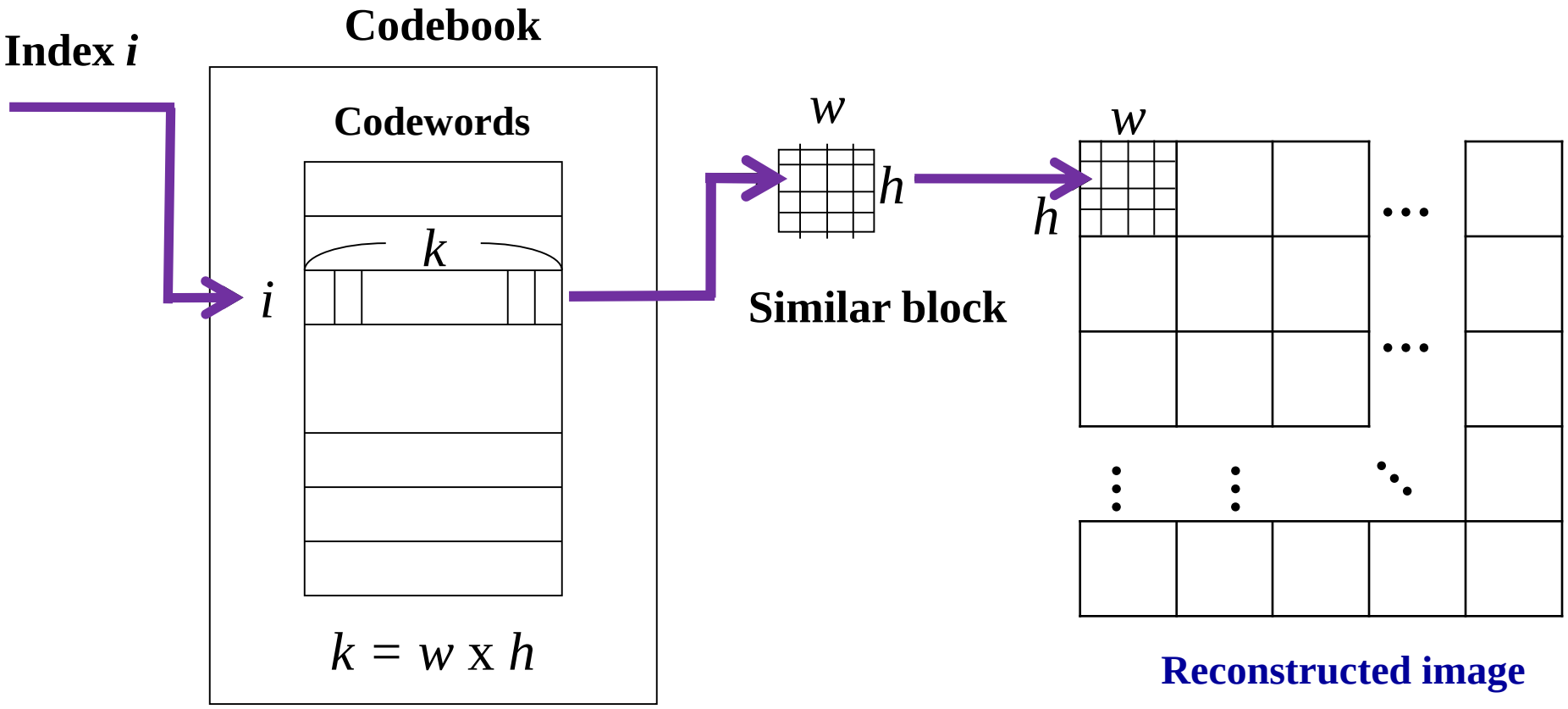
Example 1

Encoder:



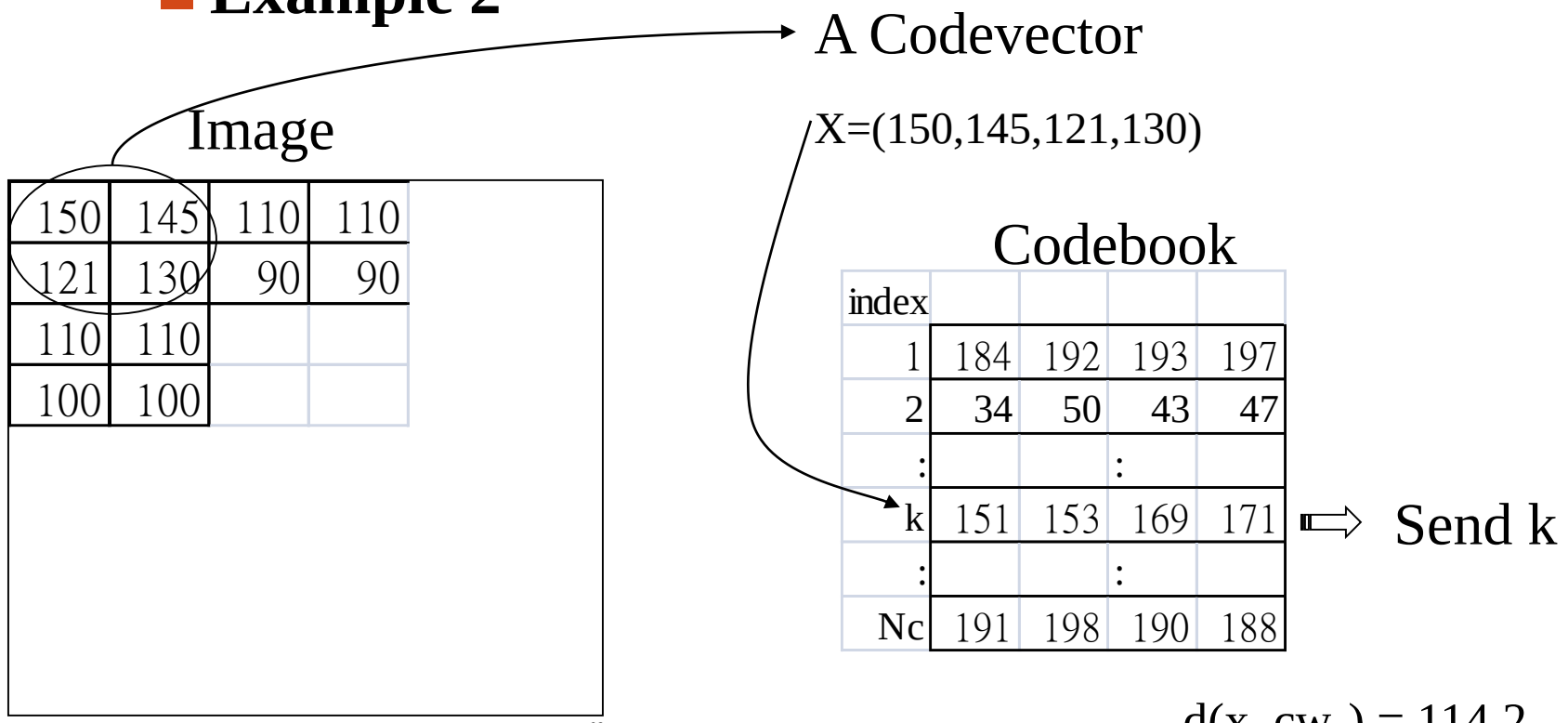
Example 1

Decoder:



Introduction to VQ

Example 2



where $d(X,\hat{X}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$ (MSE)

or $d_w(X,\hat{X}) = \frac{1}{n}\sum_{i=1}^{n} w_i(x_i - \hat{x}_i)^2$

(weighted MSE) w_i : $w\,eighting$ factor

$d(x, cw_1) = 114.2$

$d(x, cw_2) = 188.3$

$\vdots$

$d(x, cw_k) = 63.2$

$\vdots$

$d(x, cw_{Nc}) = 112.3$

Coding efficiency of VQ

- **Why VQ compress data ?**
  - Number of codevectors : N_C
  - Input vector dimension: n
  - $(\log_2 N_C)/n$ bits/pixel

index	n			
1	184	192	193	197
2	34	50	43	47
:			:	
k	151	153	169	171
:			:	
Nc	191	198	190	188

- **Example :** for a block of size 4x4, N_C =128,

  $\log_2 N_C$ =7, bit rate=7/(4×4) bits/pixel

- **Two issues in VQ :**
  - Codebook generation
  - Speed up the search

Codebook size=16 →

← Codebook size=64

Codebook size=256 →

← Codebook size=1024

Codebook Size (# of codewords)	Bits Needed to Select a Codeword	Bits per Pixel	Compression Ratio
16	4	0.25	32:1
64	6	0.375	21.33:1
256	8	0.50	16:1
1024	10	0.625	12.8:1

Codeword size=16

Codebook Generation

- Codebook initialization
  - random code
    - Randomly selected Nc codevectors from training set.
  - splitting method
    - Initial Nc=1, then Nc=2, 4, 8, 16,…
  - pairwise nearest neighbor clustering
    - Initial Nc the number of all training codevector
    - Find the nearest pair to combine into a new group
    - Compute the new center of the new group
    - Repeat until the group number is the desired Nc.

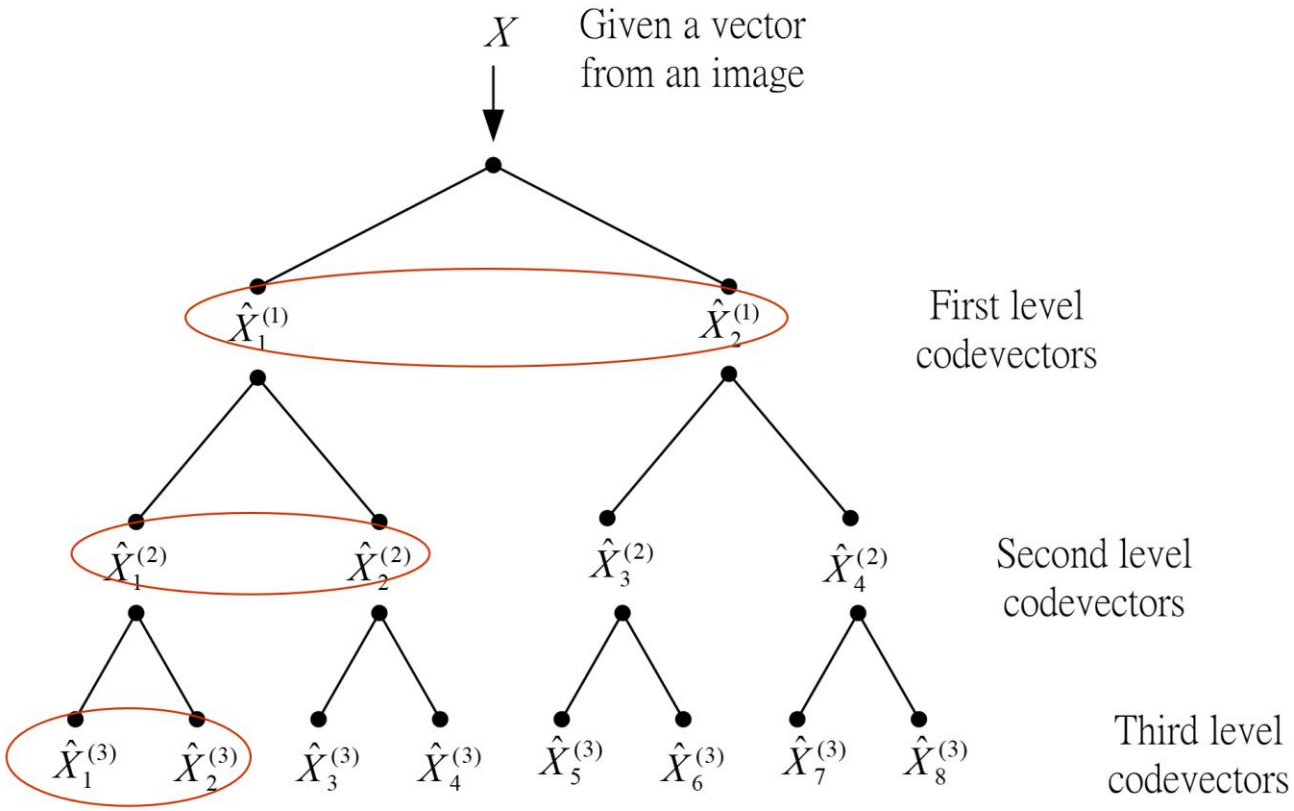
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Codebook Generation

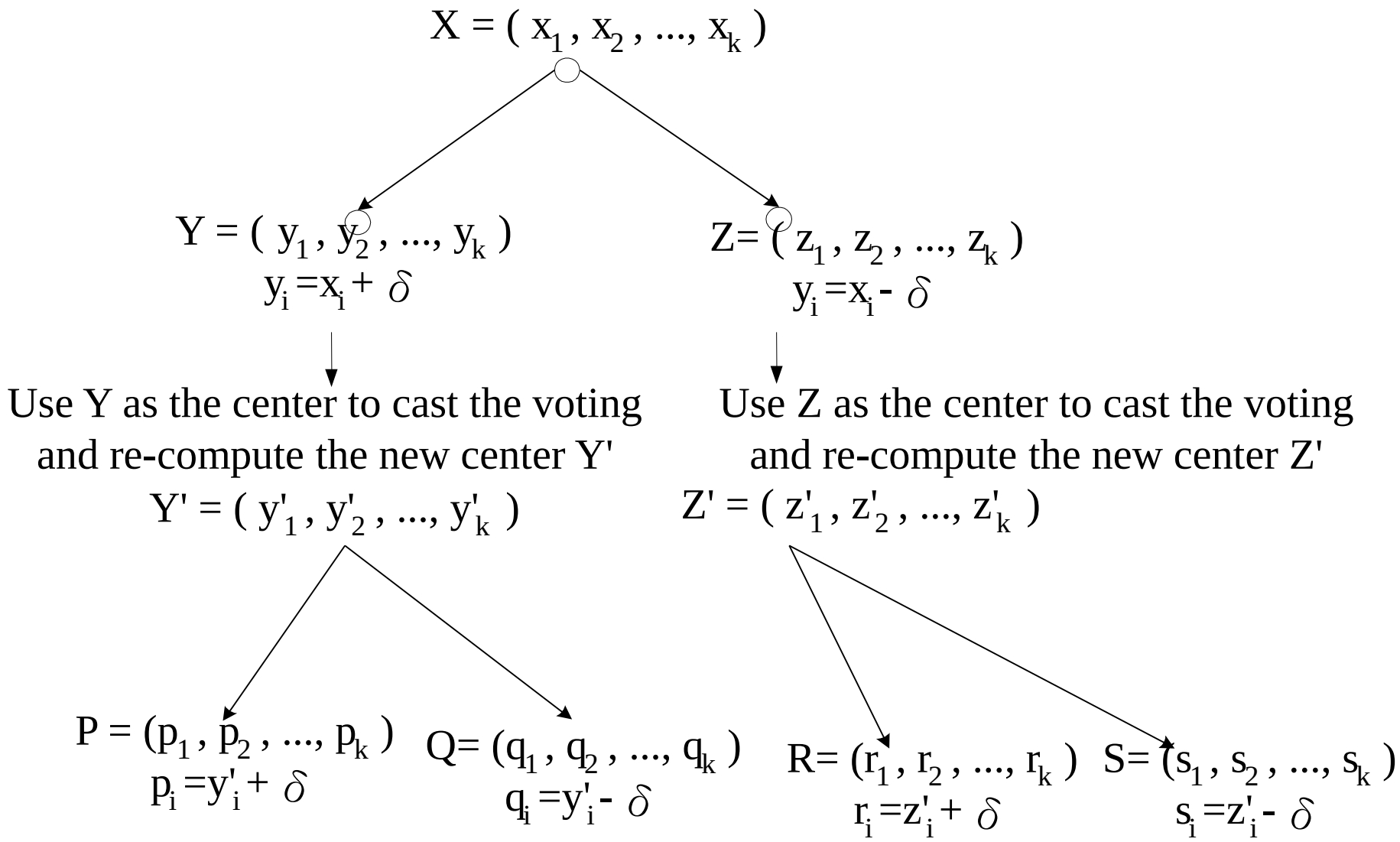
- How to measure the quality of a codebook
  - the quality of compressed image
    - PSNR
    - Human visual system
    - bpp (bits per pixel)
- Codebook size
  - small codebook results in blocking artifacts
  - large codebook achieves better quality
    - higher bit rate
    - prolong the search time

Tree Structured VQ

- TSVQ reduces the quantizer search complexity by replacing full search encoding with a sequence of tree decisions.



Cell Splitting



Example of tree growing function

- Let the displacement threshold δ be set as four and the branch number equals to two.
  - Given X = [126, 128, 126, 124].
- Step 1:

$$Y = [126, 128, 126, 124] - [4, 4, 4, 4]$$
$$= [122, 124, 122, 120]$$
$$Z = [126, 128, 126, 124] + [4, 4, 4, 4]$$
$$= [130, 132, 130, 128]$$

- Step 2: Return vector Y, Z to the caller.

Table 1: Image quality (PSNR) with codebook of size 256

512×512	LBG	$\delta = 3$		$\delta = 4$		$\delta = 5$		$\delta = 6$	
Image	PSNR	PSNR	Gain	PSNR	Gain	PSNR	Gain	PSNR	Gain
Lake	28.2476	19.6217	-8.6259	20.2762	0.6545	20.5589	0.2827	19.4946	-1.0643
Mandrill	23.7014	19.0367	-4.6647	19.3877	0.351	19.8623	0.4746	18.7432	-1.1191
Lena	31.5533	24.2782	-7.2751	26.5767	2.2985	26.5889	0.0122	25.4468	-1.1421
Pepper	30.3077	22.6068	-7.7009	21.8766	-0.7302	23.0133	1.1367	21.6787	-1.3346
Airplane	29.7907	22.3489	-7.4418	22.4328	0.0839	24.1487	1.7159	23.5129	-0.6358
Milk	34.6152	16.7874	-17.8278	18.5424	1.755	18.2859	-0.2565	17.5297	-0.7562
Averaged	29.7027	20.78	-8.9227	21.5154	0.73545	22.0763	0.56093	21.0677	-1.0087

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Table 2: Image quality (PSNR) with codebook of size 128

512×512	LBG	$\delta = 3$		$\delta = 4$		$\delta = 5$		$\delta = 6$	
Image	PSNR	PSNR	Gain	PSNR	Gain	PSNR	Gain	PSNR	Gain
Lake	27.4932	19.13	-8.3632	19.8644	0.7344	20.1012	0.2368	19.113	-0.9882
Mandrill	22.9419	19.0001	-3.9418	19.1725	0.1724	19.8227	0.6502	18.9365	-0.8862
Lena	30.5537	23.7951	-6.7586	25.8964	2.1013	26.1279	0.2315	25.4157	-0.7122
Pepper	29.7349	22.4135	-7.3214	21.3934	-1.0201	22.1824	0.789	21.1275	-1.0549
Airplane	28.2269	22.1759	-6.051	22.2205	0.0446	23.7947	1.5742	23.117	-0.6777
Milk	29.7483	17.1361	-12.6122	18.5936	1.4575	18.6153	0.0217	19.2907	0.6754
Averaged	28.1165	20.6085	-7.508	21.1901	0.58168	21.774	0.5839	21.1667	-0.6073

Tree Structured VQ

- Comparison of various kinds of Search tree

$$2^1 + 2^2 + 2^3 + 2^4 + 2^5 + 2^6 = 126$$

technique	Branches per node (m)	# of levels (nodes)tested	computations	storage
Bi-tree	**2**	**6**	**O(12n)**	**126n**
Quad-tree	**4**	**3**	**O(12n)**	**84n**
Oct-tree	**8**	**2**	**O(16n)**	**72n**
Full Search	**64**	**1**	**O(64n)**	**64n**

$NC = 64$

- Computation time is reduced from $O(nN_c)$ to $O(mn\ Log_m N_c)$

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Classified VQ (CVQ)

- A number of different codebooks are developed
  - Each is designed to encode blocks of pixels that contain a specific type of feature
- The codebook used to encode a particular block is determined by a classifier capable of differentiating between different types of features

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Classified VQ (CVQ)

- Example classes:
  - Shade blocks (no significant gradient)
  - Midrange blocks (moderate gradient, no definite edge)
  - Horizontal edge blocks
  - Vertical edge blocks
  - 45 degree or 135 degree edge blocks
  - Mixed blocks (no discernible orientation)

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Classified VQ (CVQ)

- Each classified block is encoded using the appropriate codebook
  - Codebooks can be of different sizes
- Index of the chosen codevector is sent to the receiver and is used as an entry to a look-up table
  - Each code is unique, no need to code each class separately

M/RVQ (Mean/Residual VQ)

- Means removed from the 4×4 blocks and scalar quantized.
- VQ the residual image in 4×4.
- TSVQ for the residual image to increase the efficiency
  - M/RTVQ

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Experimental Results

- Block size is 4x4
- Training set: 8 different 512x512 images
- Methods
  - 8 branches for tree structure, each level increases 3 bits
  - M/RTVQ (The mean values are quantized into 8 bits.)

M/RVQ

LBG

Conclusions

- Active research area in data compression
  - How to generate a better codebook ?
  - Fast VQ search
  - How to compress the index ?
- Similar to Huffman coding, it can be used as a postprocessing of other coding methods.

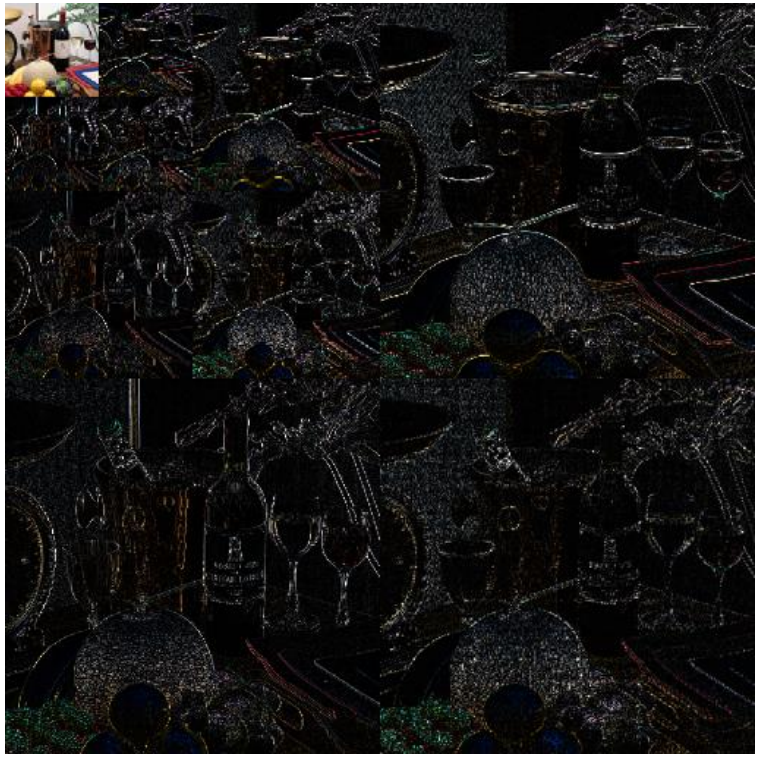
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Subband and Wavelet Coding

Wavelet History

- Application math to DSP technology
  - Morlet gave the name ’wavelet’ in 1984.
  - Daubechies constructed compactly supported biorthogonal wavelets in 1988.
  - Mallat developed DWT in 1989.
  - Sweldens proposed Lifting Scheme in 1996.
  - JPEG2000 adopted DWT as the core transform.
- Target
  - Higher spatial-frequency performance and finer scalability than DCT
- Critical issues of hardware implementation
  - Much more required computation than DCT
  - Frame-based coding requires huge buffer

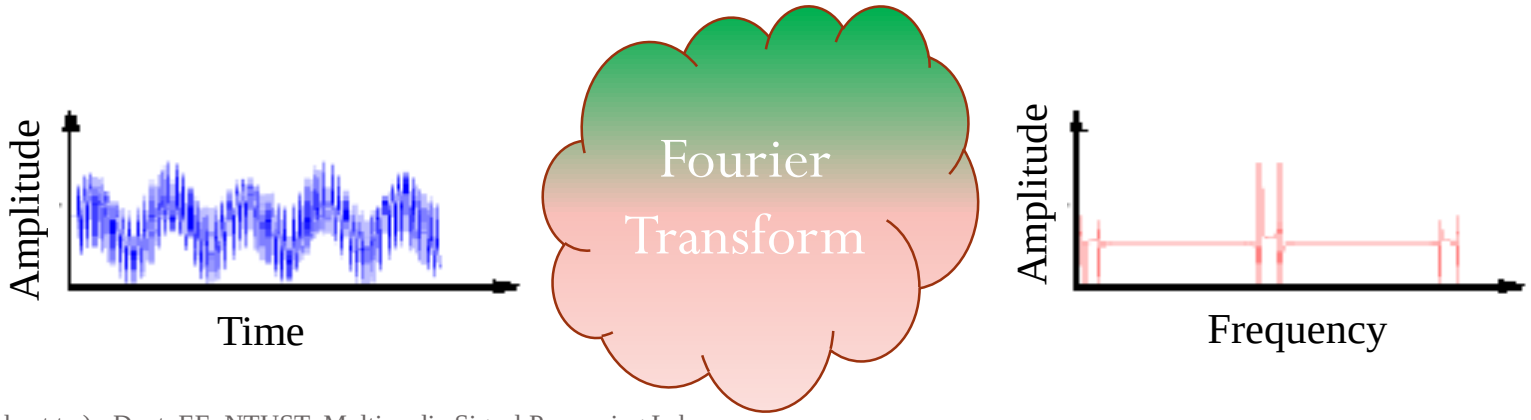
Wavelet Transform—A Sample



3rd level decomposition

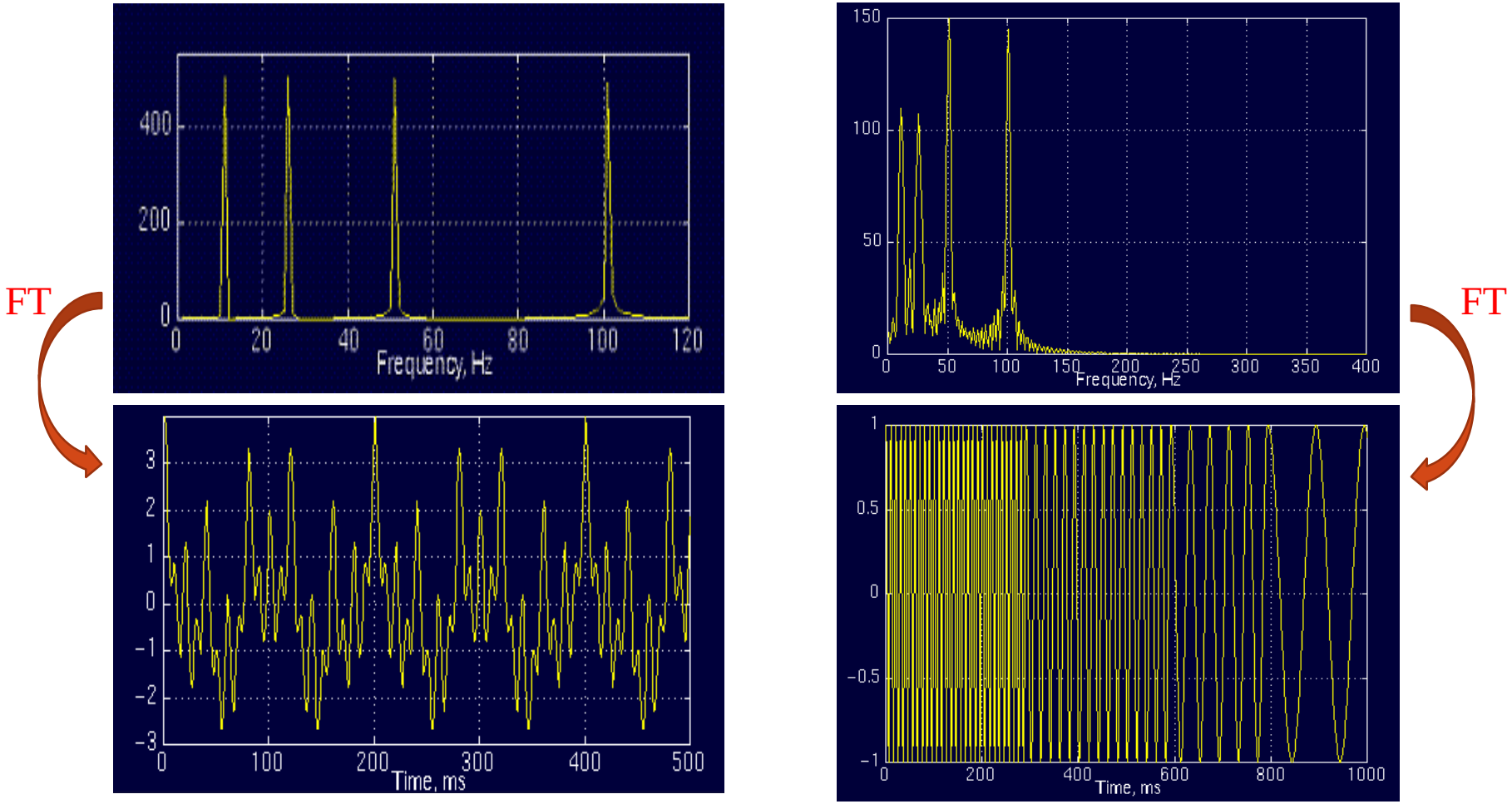
Fourier Transform

- The frequency spectrum of the signal shows what frequencies exist in the signal
- FT
  - Frequency domain ☺
  - Time domain ☹
- No frequency information is available in time-domain
- No time information is available in frequency-domain signal



Comparison of two examples

- Two spectrums are similar!
- Four spectral components at exactly the same frequencies
- The corresponding time domain signals are not even close



Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Nothing More, Nothing Less

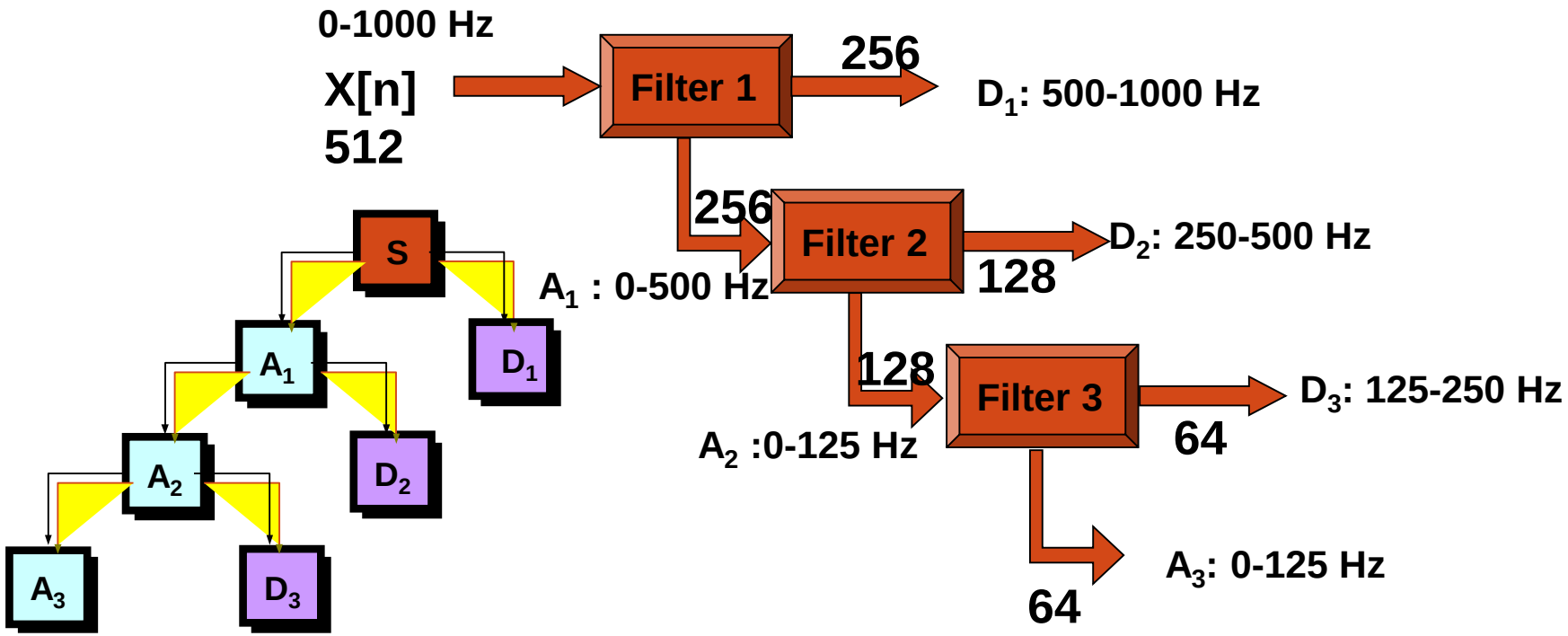
- FT only gives what frequency components exist in the signal
- The time and frequency information can not be seen at the same time
- Time-frequency representation of the signal is needed

Most of Transportation Signals are Non-stationary.
(We need to know whether and also when an incident was happened.)

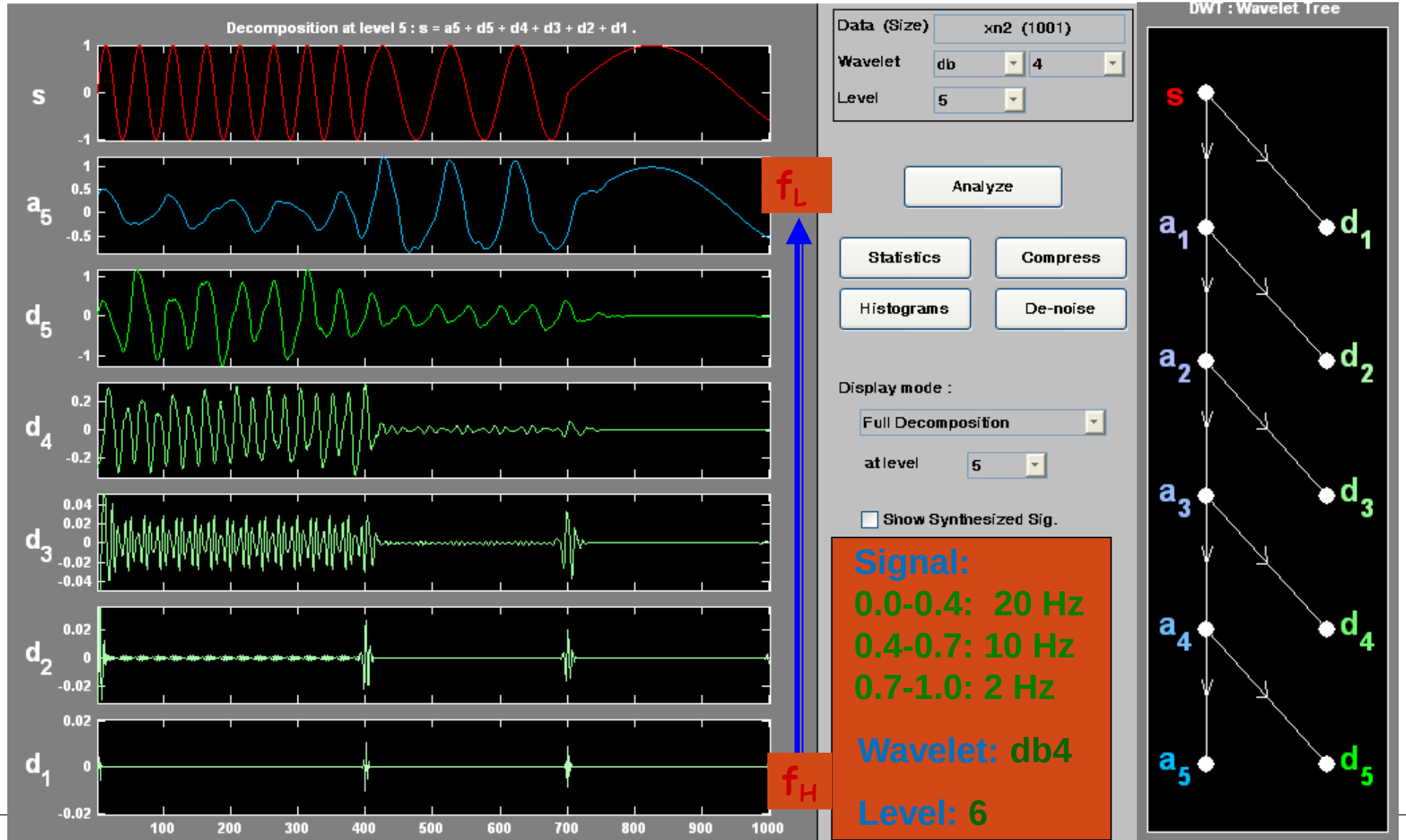
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Subband Coding Algorithm

- Halves the time resolution
  - Only half number of samples resulted
- Doubles the frequency resolution
  - The spanned frequency band halved



Decomposing Non-Stationary Signals (1)



DWT 5-3 and 9-7 Filters

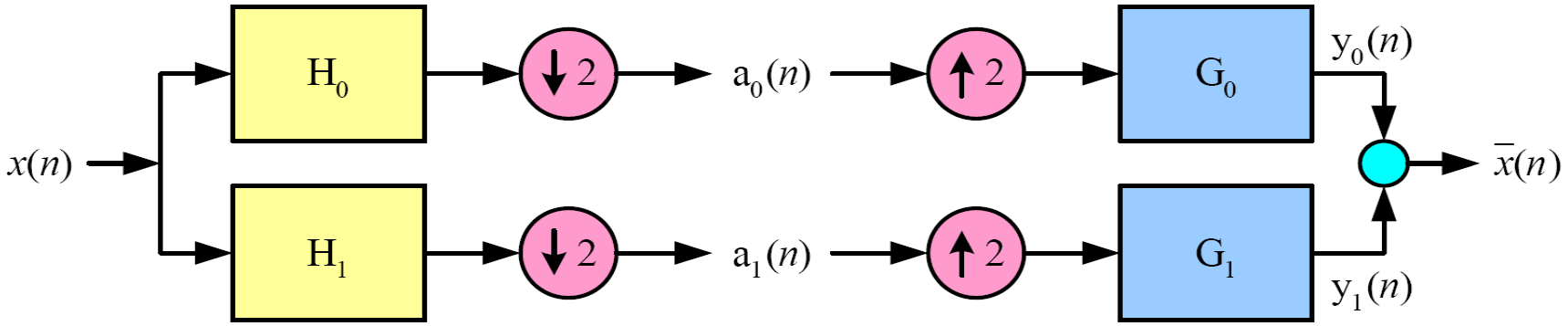
- Wavelet 5-3 filter for lossless coding
  - Integer arithmetic, low implementation complexity

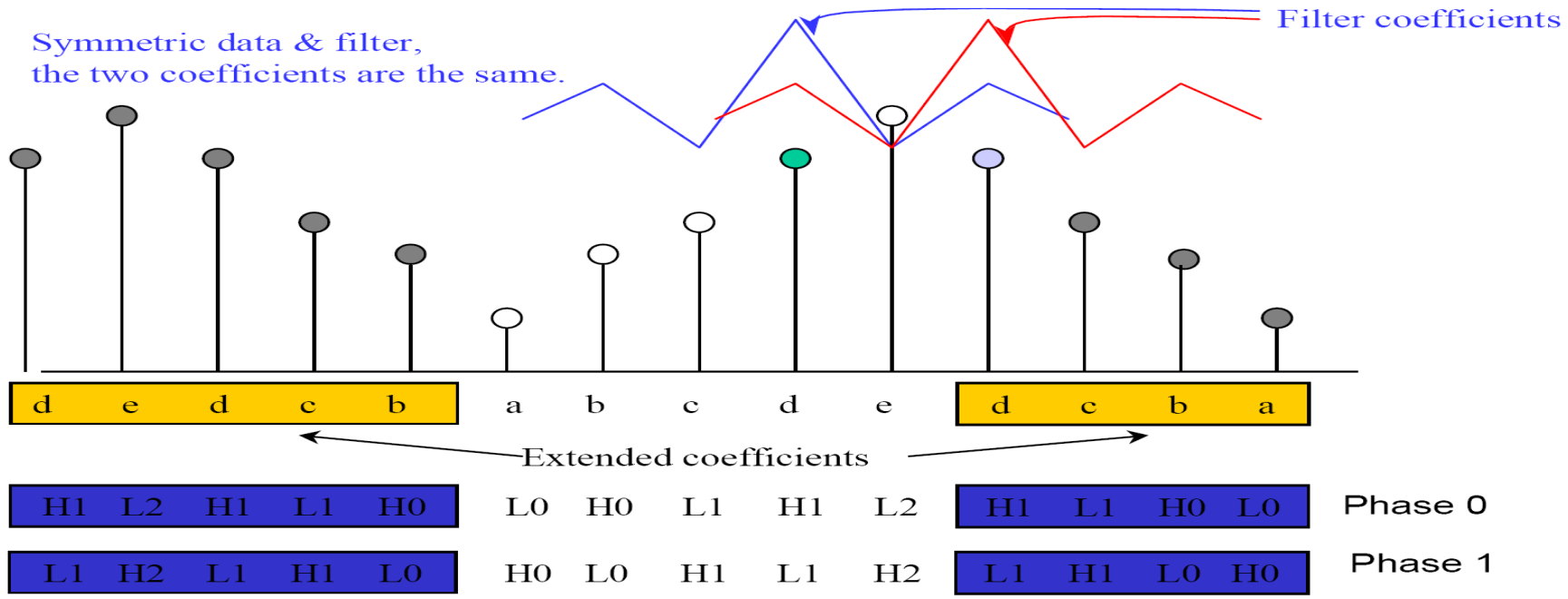
i	Ho	H1
0	6/8	1
1	2/8	-1/2
2	-1/8	0

- Wavelet 9-7 filter for lossy compression
  - Best performance at low bit rate
  - Relatively high implementation complexity, in particular for hardware

i	Ho	H1
0	0.6029490182363579	1.115087052456994
1	0.2668641184428723	-0.5912717631142470
2	-0.07822326652898785	-0.05754352622849957
3	-0.01686411844287495	0.09127176311424948
4	0.02674872741080976	

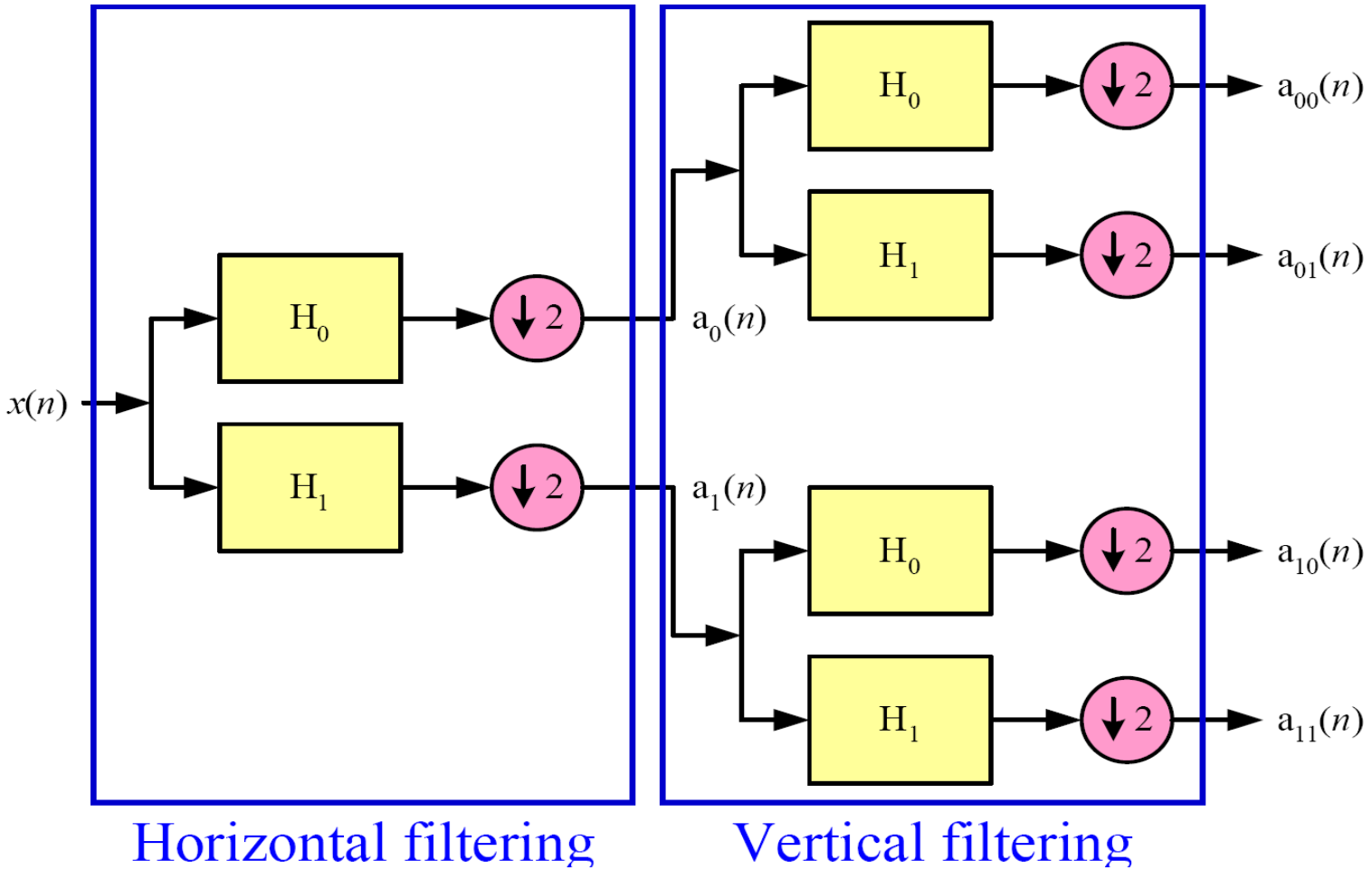
Discrete Wavelet Transform—1D





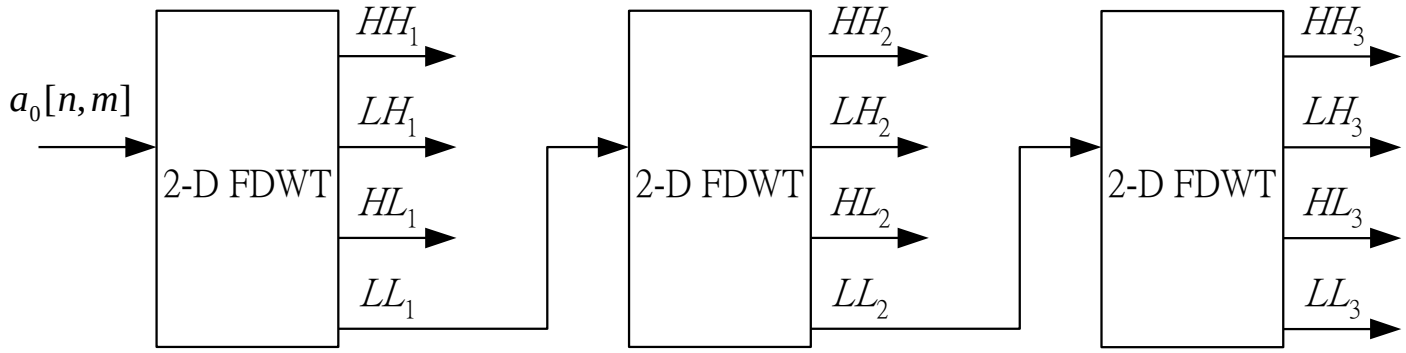
Rule for downsampling:

- Low pass result: remove the odd columns or rows
- High pass result: remove the even columns or rows

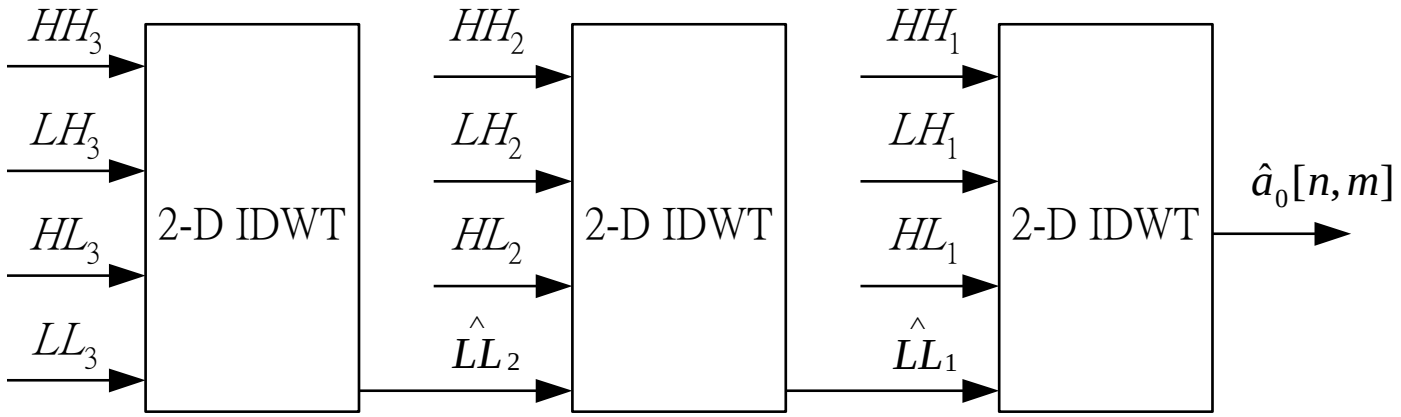


Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

3-level DWT procedure

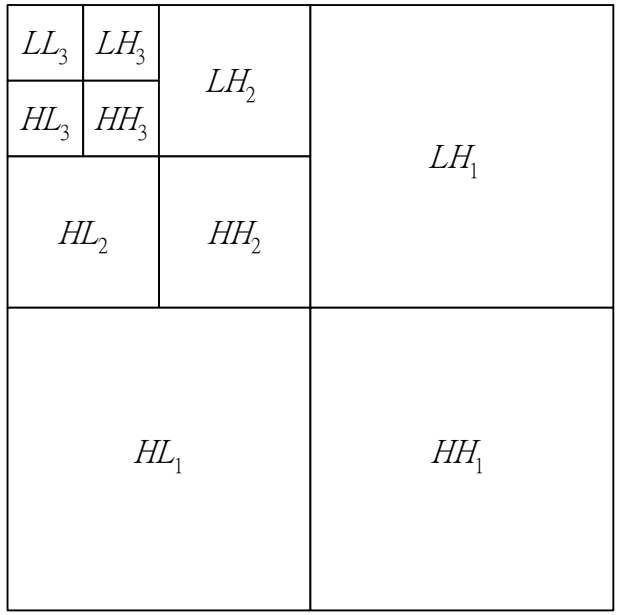


3-level 2-D FDWT procedure



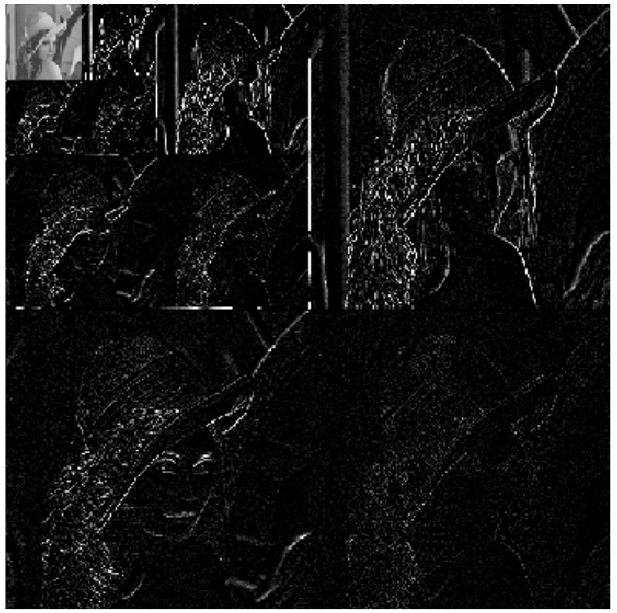
3-level 2-D IDWT procedure

3-level DWT result



3-level 2-D FDWT

Original Lena image

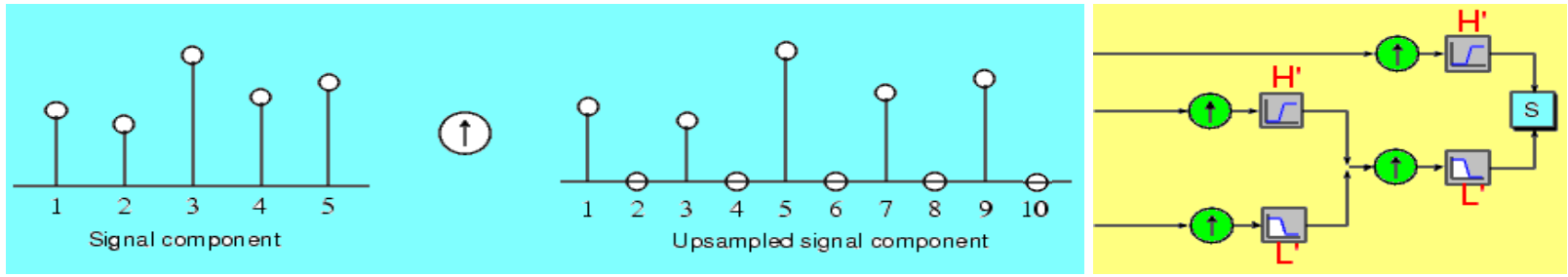


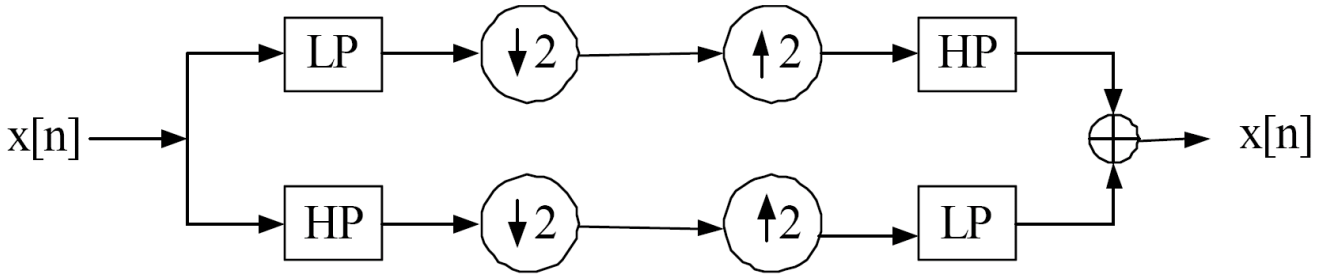
3-level 2-D FDWT

Inverse DWT

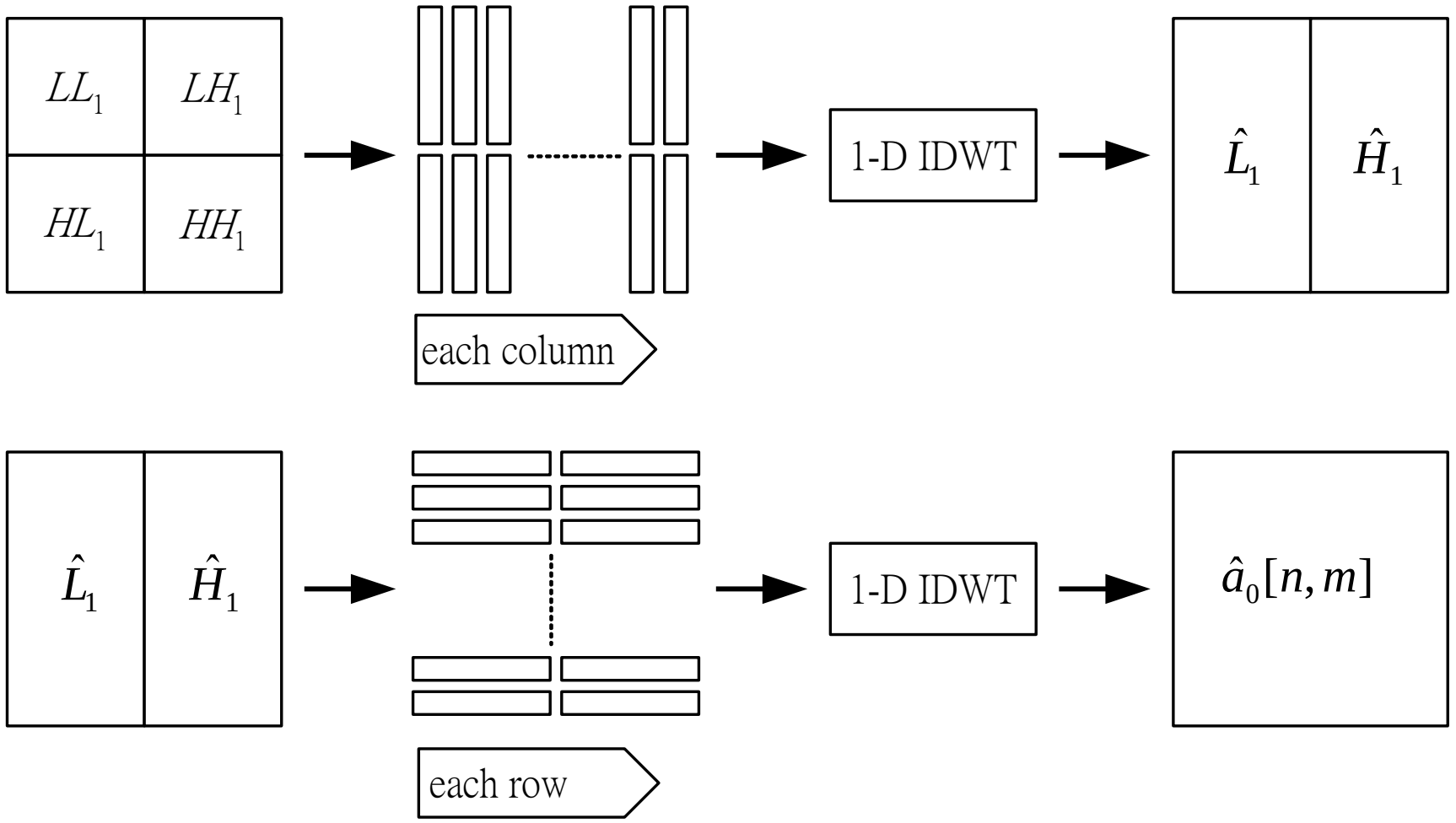
- Lowpass: Lengthening a signal component by inserting zeros between samples; ends with zero (upsampling); then processed with new highpass filter
- Highpass: Lengthening a signal component by inserting zeros; between samples; starts with zero (upsampling); then processed with new lowpass filter
- The results obtained by highpass filtering and lowpass filtering are summed together to obtain the reconstructed image

Lowpass case





2-D IDWT procedure



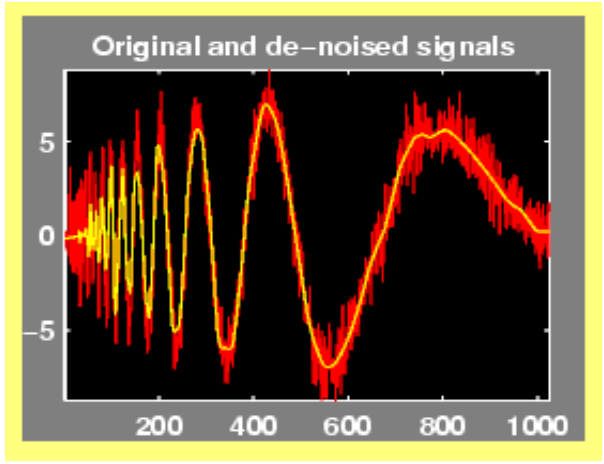
Wavelet Applications

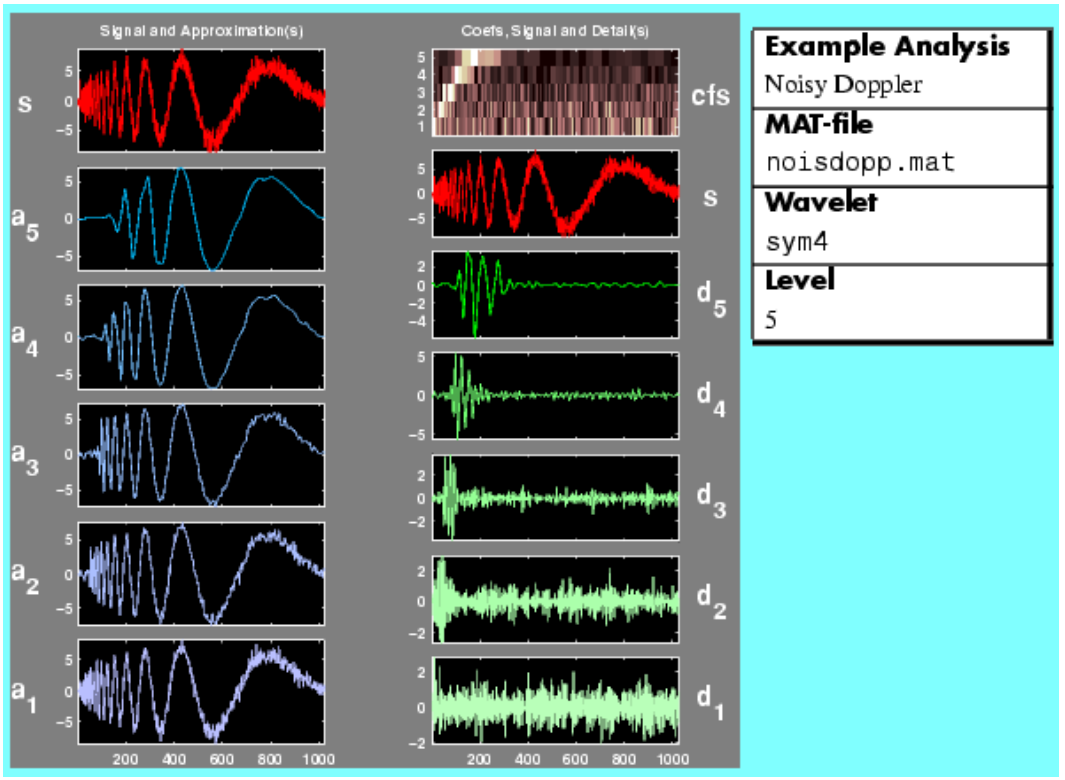
- Typical Application Fields
  - Astronomy, acoustics, nuclear engineering, sub-band coding, signal and image processing, neurophysiology, music, magnetic resonance imaging, speech discrimination, optics, fractals, turbulence, earthquake-prediction, radar, human vision, and pure mathematics applications
- Sample Applications
  - Identifying pure frequencies
  - De-noising signals
  - Detecting discontinuities and breakdown points
  - Detecting self-similarity
  - Compressing images

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

De-noising Signals

- Highest frequencies appear at the start of the original signal
- Approximations appear less and less noisy
- Also lose progressively more high-frequency information.
- In A_5, about the first 20% of the signal is truncated





Discrete Cosine Transform

Transform Coding

- Spatial image data (image or motion-compensated residual image) are transformed into a different representation, transform domain.
  - Make the image data easy to be compressed
- Techniques:
  - Discrete Cosine Transform (DCT)
    - Usually applied to small regular blocks of image, ex. 8 x 8 squares
    - JPEG, H.26x, MPEG-x
  - Discrete Wavelet Transform (DWT)
    - Usually applied to larger image section, ex. Tiles, or to complete image
    - JPEG 2000, MPEG-4 still texture

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Basic Transformation Forms

- 2-D forward transforms

$$T(u,v) = \sum_{x=0}^{N-1}\sum_{y=0}^{N-1} g(x,y) f(x,y,u,v),$$

Original signal

2-D inverse transforms

$$g(x,y) = \sum_{u=0}^{N-1}\sum_{v=0}^{N-1} T(u,v) i(x,y,u,v),$$

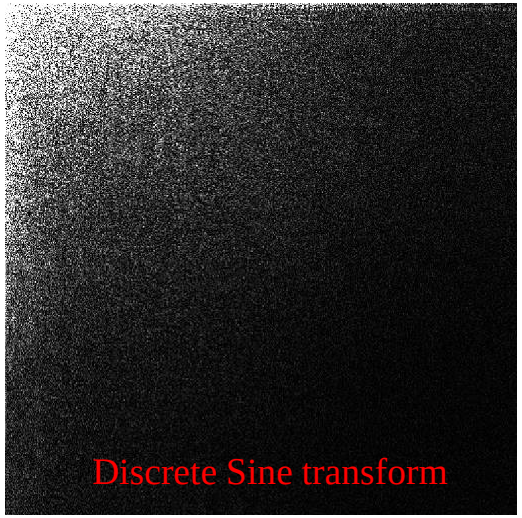
Transformed coefficients

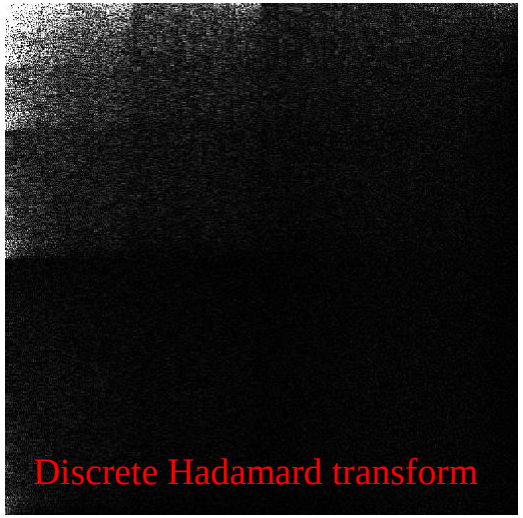
where f (x, y, u, v) and i(x, y, u, v) are referred to as the forward and inverse transformation kernels, respectively.

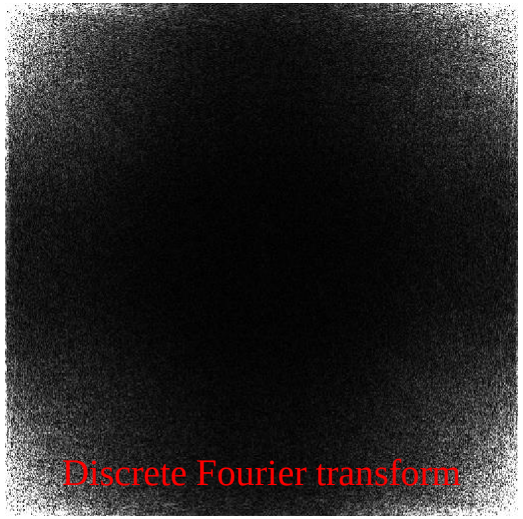
- DFT, DHT, DCT, DST, KLT, DWT, …..

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Compaction Efficiency For Various Image Transforms





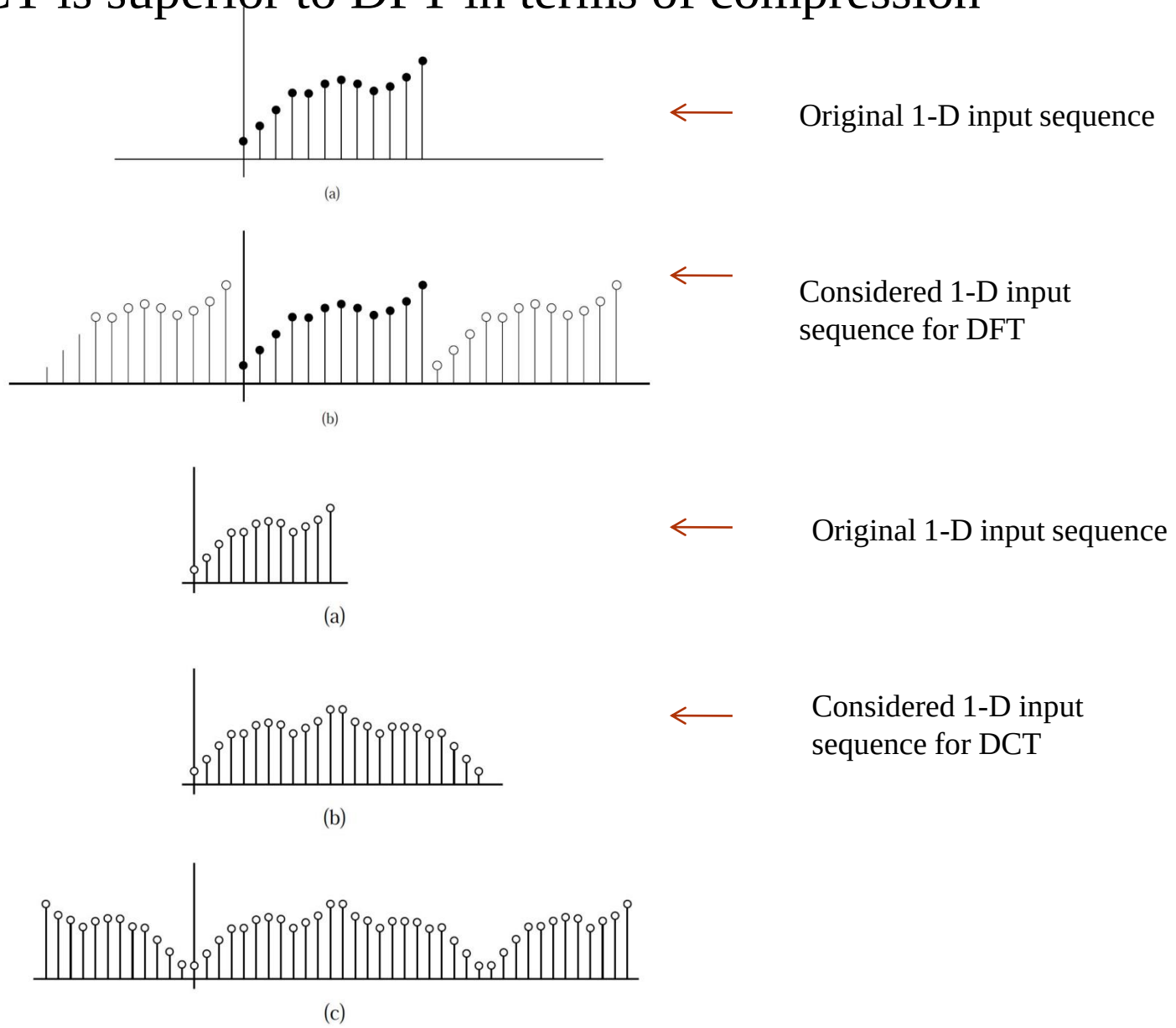


Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

DCT-Based Coding

- Optimal transform is KLT, but
  - KLT is image dependent
  - complex computing complexity
- DCT-based coding,
  - is image independent, unlike KLT for highly correlated image data,
  - DCT compaction efficiency is close to KLT.
  - Computations of DCT can be performance with fast algorithms which can be easily implemented on parallel architectures.

Reason why DCT is superior to DFT in terms of compression



Original 1-D input sequence

Considered 1-D input sequence for DFT

Original 1-D input sequence

Considered 1-D input sequence for DCT

Implementation of the DCT

- DCT-based codecs use a two-dimensional version of the transform.
- The 2-D DCT and its inverse (IDCT) of an N x N block are shown below:

  - 2-D DCT: $F(u,v) = \frac{2}{N} C(u)C(v) \sum_{y=0}^{N-1} \sum_{x=0}^{N-1} f(x,y) \cos[\frac{(2x+1)u\pi}{2N}] \cos[\frac{(2y+1)v\pi}{2N}]$
  - 2-D IDCT: $f(x,y) = \frac{2}{N} \sum_{v=0}^{N-1} \sum_{u=0}^{N-1} C(u)C(v)F(u,v) \cos[\frac{(2x+1)u\pi}{2N}] \cos[\frac{(2y+1)v\pi}{2N}]$

$$C(u), C(v) = \frac{1}{\sqrt{2}} \quad for \;\; u, v = 0$$
$$C(u), C(v) = 1 \;\; otherwise$$

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

$$F(i,j) = \frac{2}{N} C(i)C(j) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\left[\frac{(2x+1)i\pi}{2N}\right] \cos\left[\frac{(2y+1)j\pi}{2N}\right]$$

f(x,y)

	0	1
0	8	5
1	3	4

F(i,j)

	0	1
0	10	
1		

$$F(0,0) = \frac{2}{2} \times \frac{1}{\sqrt{2}} \times \frac{1}{\sqrt{2}} \times$$

$$\begin{aligned}
&8 \times \cos\left(\frac{(2\times 0+1)\times 0\times \pi}{2\times 2}\right) \times \cos\left(\frac{(2\times 0+1)\times 0\times \pi}{2\times 2}\right) + \\
&5 \times \cos\left(\frac{(2\times 0+1)\times 0\times \pi}{2\times 2}\right) \times \cos\left(\frac{(2\times 1+1)\times 0\times \pi}{2\times 2}\right) + \\
&3 \times \cos\left(\frac{(2\times 1+1)\times 0\times \pi}{2\times 2}\right) \times \cos\left(\frac{(2\times 0+1)\times 0\times \pi}{2\times 2}\right) + \\
&4 \times \cos\left(\frac{(2\times 1+1)\times 0\times \pi}{2\times 2}\right) \times \cos\left(\frac{(2\times 1+1)\times 0\times \pi}{2\times 2}\right)
\end{aligned}$$

= 10

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

IDCT – 2D

$$f(x,y) = \frac{2}{N}\sum_{i=0}^{N-1}\sum_{j=0}^{N-1} C(i)C(j)F(i,j)\cos\left[\frac{(2x+1)i\pi}{2N}\right]\cos\left[\frac{(2y+1)j\pi}{2N}\right]$$

f(x,y)

	0	1
0	8	
1		

F(i,j)

	0	1
0	10	1
1	3	2

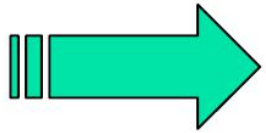
$$f(0,0) = \frac{2}{2}\times \begin{aligned} &\frac{1}{\sqrt{2}}\times\frac{1}{\sqrt{2}}\times 10\times\cos\left(\frac{(2\times 0+1)\times 0\times\pi}{2\times 2}\right)\times\cos\left(\frac{(2\times 0+1)\times 0\times\pi}{2\times 2}\right)+\\ &\frac{1}{\sqrt{2}}\times 1\times 1\times \cos\left(\frac{(2\times 0+1)\times 0\times\pi}{2\times 2}\right)\times\cos\left(\frac{(2\times 0+1)\times 1\times\pi}{2\times 2}\right)+\\ &1\times\frac{1}{\sqrt{2}}\times 3\times \cos\left(\frac{(2\times 0+1)\times 1\times\pi}{2\times 2}\right)\times\cos\left(\frac{(2\times 0+1)\times 0\times\pi}{2\times 2}\right)+\\ &1\times 1\times 2\times \cos\left(\frac{(2\times 0+1)\times 1\times\pi}{2\times 2}\right)\times\cos\left(\frac{(2\times 0+1)\times 1\times\pi}{2\times 2}\right) \end{aligned}$$

= 8

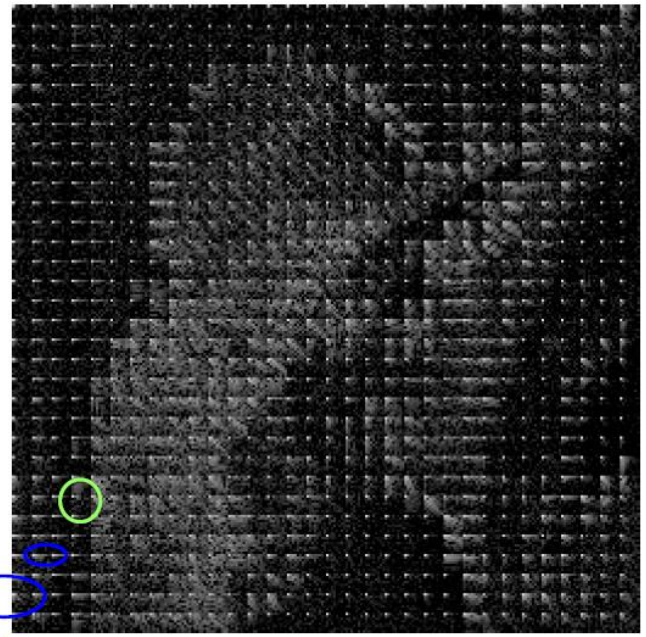
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

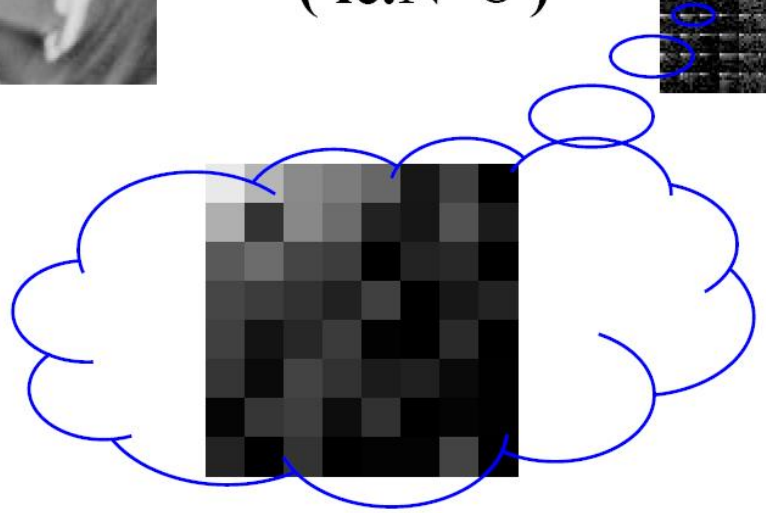
Discrete Cosine Transform

DCT
block size 8 x 8

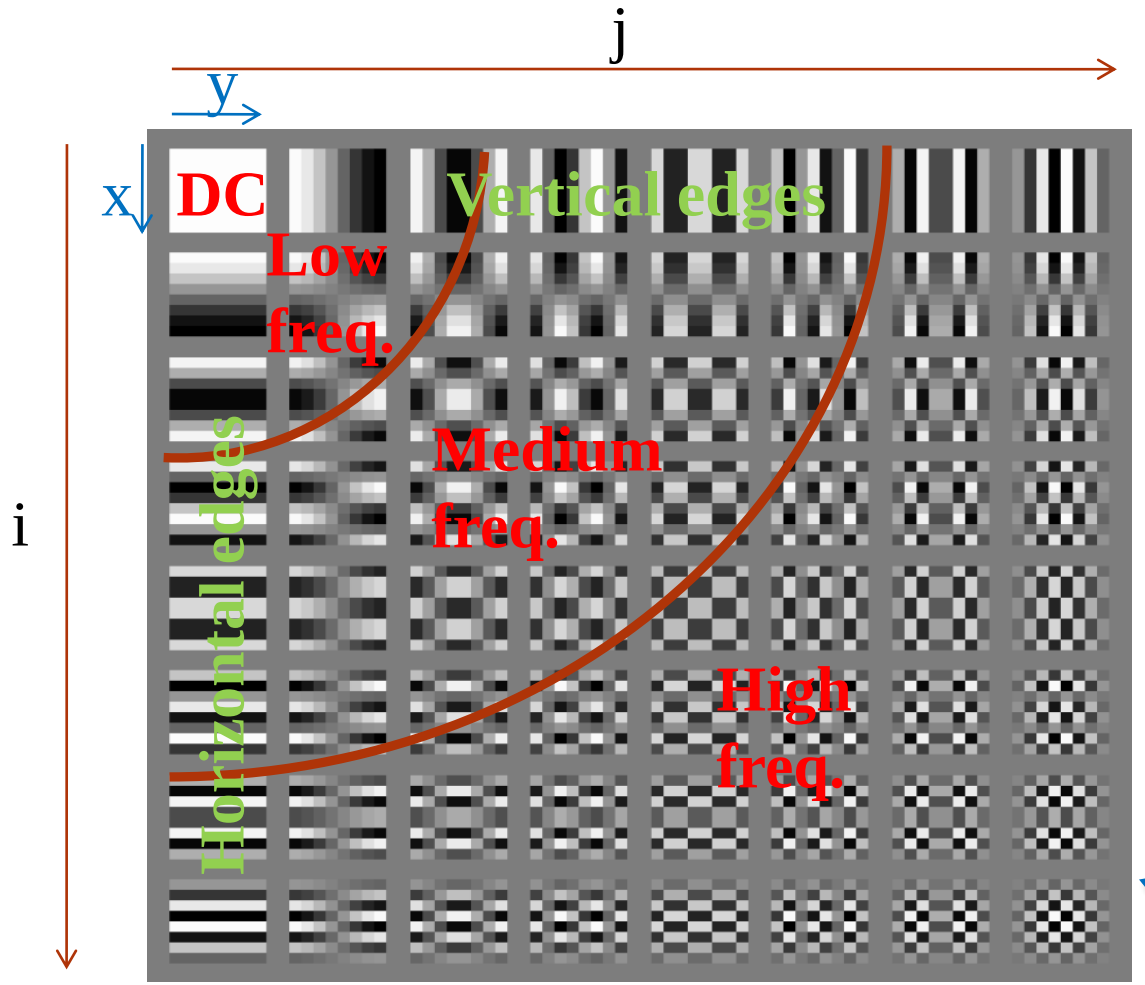


(ie.N=8)





DCT Basis



$$F(i,j) = \frac{2}{N} C(i) C(j) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\left[\frac{(2x+1)i\pi}{2N}\right] \cos\left[\frac{(2y+1)j\pi}{2N}\right]$$

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

1D-DCT

- 1-D
- 8-point DCT

$$F(u) = C(u)\sum_{m=0}^{N-1} f(m)\cos\left[\frac{(2m+1)u\pi}{16}\right]$$

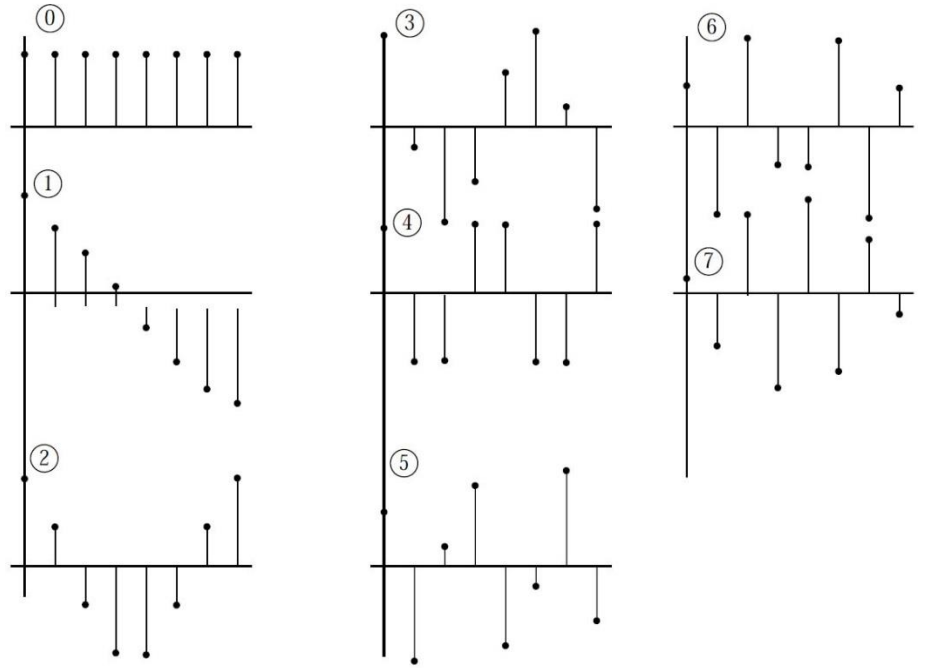
$u = 0,1,\ldots,7.$

$$f(m) = \sum_{u=0}^{N-1} C(u)F(u)\cos\left[\frac{(2m+1)u\pi}{16}\right]$$

$m = 0,1,\ldots,7.$

where

$$C(0) = \sqrt{\frac{1}{8}}, \quad C(u) = \sqrt{\frac{2}{8}}, \quad u = 1,2,\ldots,M-1$$



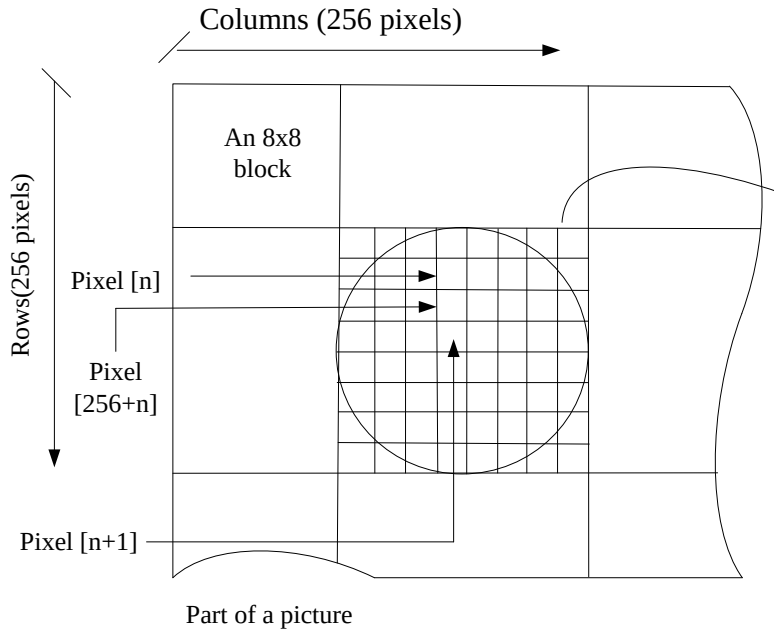
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

2-D DCT using a 1-D DCT Pair

- One of the properties of the 2-D DCT is that it is separable, meaning that it can be separated into a pair of 1-D DCTs.
- To obtain the 2-D DCT of a block, a 1-D DCT is first performed on the rows of the block then a 1-D DCT is performed on the columns of the resulting block.
- The same applies to the IDCT.
- This process is illustrated on the following slide.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

2-D DCT using a 1-D DCT Pair



Raw Block of pixels

128	127	130	128	134	130	128	128
128	127	130	128	134	130	128	128
125	126	130	127	130	131	128	127
124	128	127	126	129	130	127	127
131	132	130	129	132	128	129	131
129	131	134	131	133	129	131	131
129	134	134	136	136	137	134	132
133	135	136	138	137	140	136	132

Result after DCT by rows

356	-1	-4	1	1	-3	1	3
356	-1	-4	1	1	-3	1	3
352	-3	-4	0	-3	-1	2	1
399	-3	-3	0	-3	-3	0	-1
358	1	1	-1	1	-3	-2	1
370	0	-3	-3	-1	-1	-1	2
378	-3	-6	0	-3	-2	-1	-2
383	-1	-6	2	-3	0	0	-3

Final result after DCT by columns

102	-4	-11	0	-4	-6	0	1
-12	-1	1	0	2	-2	2	4
11	-1	-5	2	3	0	1	-1
0	2	2	-3	3	-2	-2	3
0	1	1	1	0	-1	-1	-2
-4	-3	-3	-1	-3	-1	0	-2
-2	0	-1	-2	-1	1	1	0
2	0	1	0	0	-1	-1	-1

Fast Algorithms For The DCT

64x64

- An F(i,j) requires 64 multiplications and 64 additions.
- 4096 multiply accumulate operations are needed for each 8x8 block.
- Use the row-column decomposition, only 16 1-D DCTs (8 for row and 8 for column) is needed (total 1024 multiply-accumulate operations)

8x8x8x2

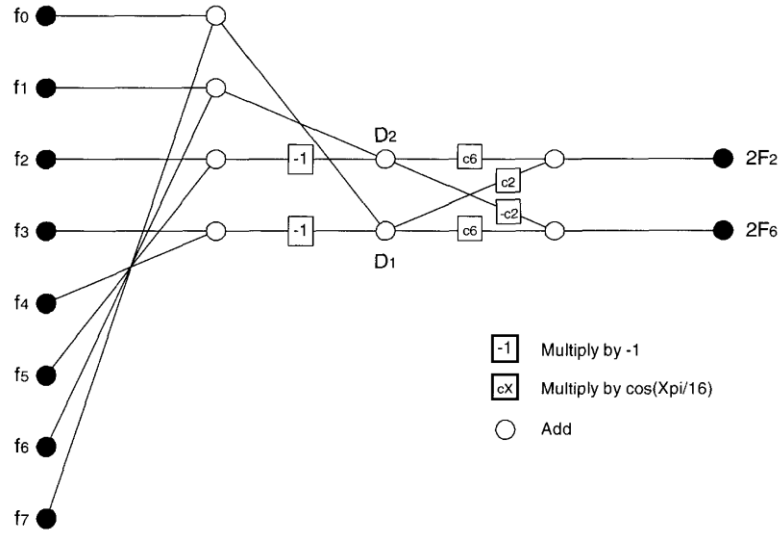
$$F(i,j) = \frac{2}{N} C(i)C(j) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\left[\frac{(2x+1)i\pi}{2N}\right] \cos\left[\frac{(2y+1)j\pi}{2N}\right]$$

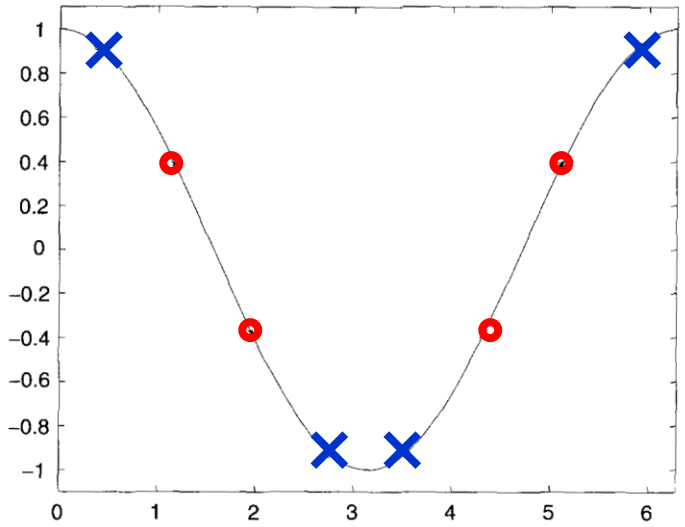
Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Fast DCT algorithms

$$F(k)=\frac{c(k)}{2}\sum_{n=0}^{7}f(n)\cos\left(\frac{2n+1}{16}\cdot k\pi\right),\ c(0)=\frac{1}{\sqrt{2}},\ c(x)=1$$

- Flowgraph algorithm





$$F_2=\frac{1}{2}\sum_{i=0}^{7}f_i\cos\left(\frac{(2i+1).2\pi}{16}\right)$$

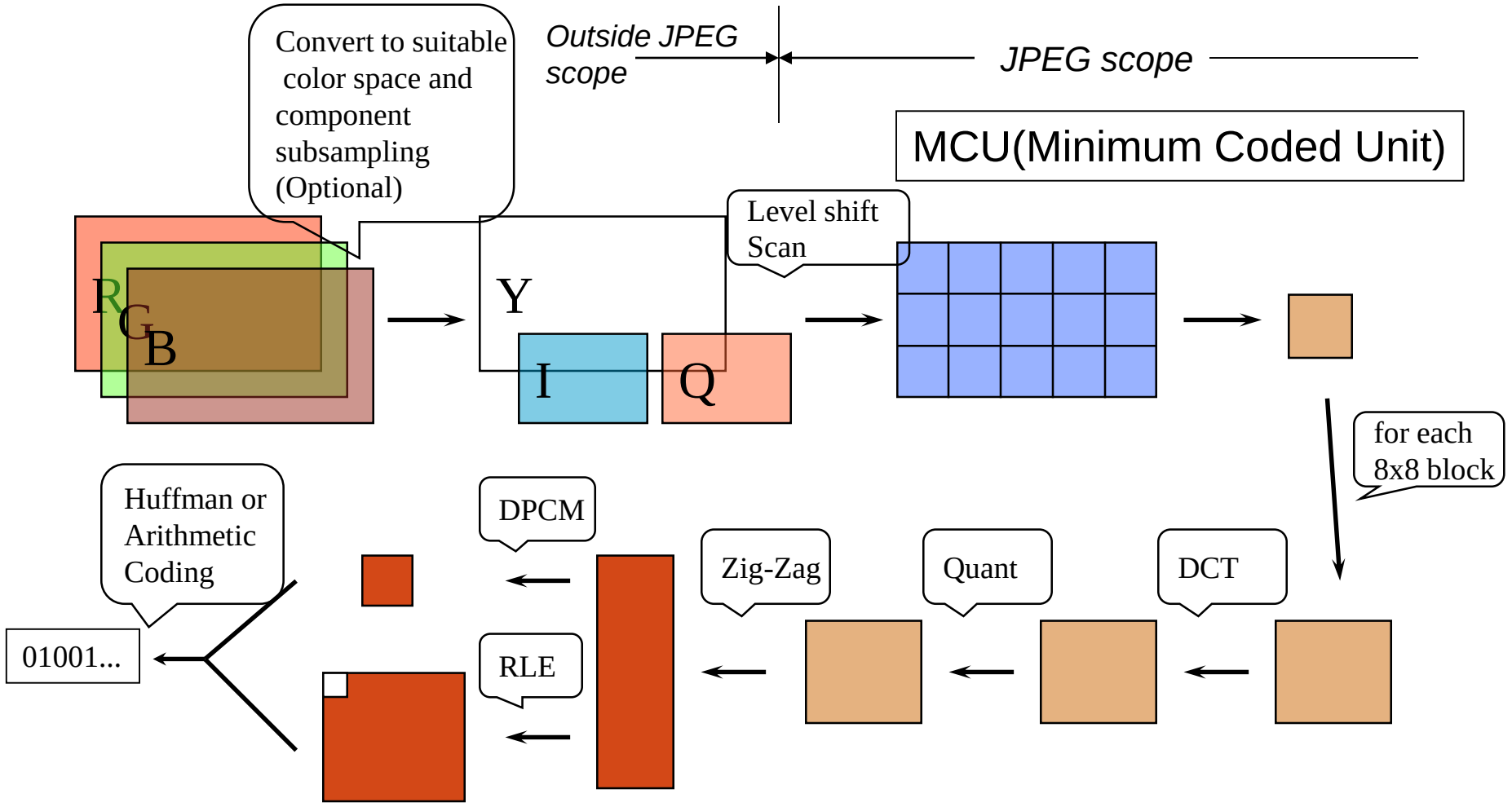
$$=\frac{1}{2}\left[f_o\cos\left(\frac{\pi}{8}\right)+f_1\cos\left(\frac{3\pi}{8}\right)+f_2\cos\left(\frac{5\pi}{8}\right)+f_3\cos\left(\frac{7\pi}{8}\right)+f_4\cos\left(\frac{9\pi}{8}\right)+f_5\cos\left(\frac{11\pi}{8}\right)+f_6\cos\left(\frac{13\pi}{8}\right)+f_7\cos\left(\frac{15\pi}{8}\right)\right]$$

$$2F_2=\left[(f_0-f_4+f_7-f_3)\cdot\cos\left(\frac{\pi}{8}\right)+(f_1-f_2-f_5+f_6)\cdot\cos\left(\frac{3\pi}{8}\right)\right]$$

$$D_1=(f_0-f_4+f_7-f_3)\quad and\quad D_2=(f_1-f_2+f_5+f_6)$$

$$2F_2=\left[D_1\cos\left(\frac{\pi}{8}\right)+D_2\cos\left(\frac{3\pi}{8}\right)\right]\quad and\quad 2F_6=\left[D_1\cos\left(\frac{3\pi}{8}\right)+D_2\cos\left(\frac{\pi}{8}\right)\right]$$

Application to image compression-JPEG (baseline)



Discrete Cosine Transform (DCT)

Example of 2-D 8x8 DCT/IDCT:

$$\begin{bmatrix} 139 & 144 & 149 & 153 & 155 & 155 & 155 & 155 \\ 144 & 151 & 153 & 156 & 159 & 156 & 156 & 156 \\ 150 & 155 & 160 & 163 & 158 & 156 & 156 & 156 \\ 159 & 161 & 162 & 160 & 160 & 159 & 159 & 159 \\ 159 & 160 & 161 & 162 & 162 & 155 & 155 & 155 \\ 161 & 161 & 161 & 161 & 160 & 157 & 157 & 157 \\ 162 & 162 & 161 & 163 & 162 & 157 & 157 & 157 \\ 162 & 162 & 161 & 161 & 163 & 158 & 158 & 158 \end{bmatrix} \xrightarrow{\text{-128 then DCT}} \begin{bmatrix} 236 & -1 & -12 & -5 & 2 & -2 & -3 & 1 \\ -23 & -17 & -6 & -3 & -3 & 0 & 0 & -1 \\ -11 & -9 & -2 & 2 & 0 & -1 & -1 & 0 \\ -7 & -2 & 0 & 1 & 1 & 0 & 0 & 0 \\ -1 & -1 & 1 & 2 & 0 & -1 & 1 & 1 \\ 2 & 0 & 2 & 0 & -1 & 1 & 1 & -1 \\ -1 & 0 & 0 & -1 & 0 & 2 & 1 & -1 \\ -3 & 2 & -4 & -2 & 2 & 1 & -1 & 0 \end{bmatrix}$$

Pixel (Spatial) Domain Frequency Domain

Obviously, the “energy” is compacted toward the low frequency.

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Quantization

Example of Quantization

$$\begin{bmatrix} 236 & -1 & -12 & -5 & 2 & -2 & -3 & 1 \\ -23 & -17 & -6 & -3 & -3 & 0 & 0 & -1 \\ -11 & -9 & -2 & 2 & 0 & -1 & -1 & 0 \\ -7 & -2 & 0 & 1 & 1 & 0 & 0 & 0 \\ -1 & -1 & 1 & 2 & 0 & -1 & 1 & 1 \\ 2 & 0 & 2 & 0 & -1 & 1 & 1 & -1 \\ -1 & 0 & 0 & -1 & 0 & 2 & 1 & -1 \\ -3 & 2 & -4 & -2 & 2 & 1 & -1 & 0 \end{bmatrix} \div \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix} = \begin{bmatrix} 15 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

DCT matrix

Quantization matrix (Quantization table)

← Also called Q-table

Quantized DCT matrix

Due to the human vision is not sensitive to the high frequency coefficient, the quantization coefficient will be much larger in high frequency position.

Quality Factor

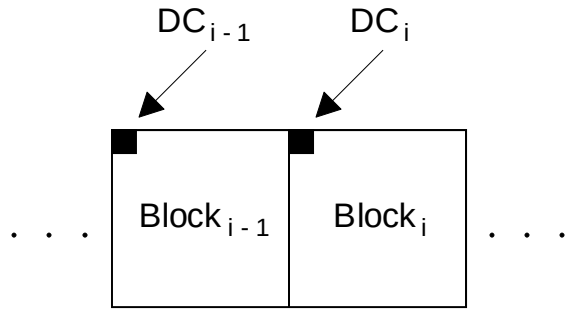
- Quality Factor (QF) is ranged between 1 to 99
- It is used to scale the quantization table by a scaling factor (SF) calculated as

$$\begin{cases} SF = \dfrac{5000}{QF}, & if\ QF \le 50 \\ SF = 200 - QF * 2, & if\ QF > 50 \end{cases}$$

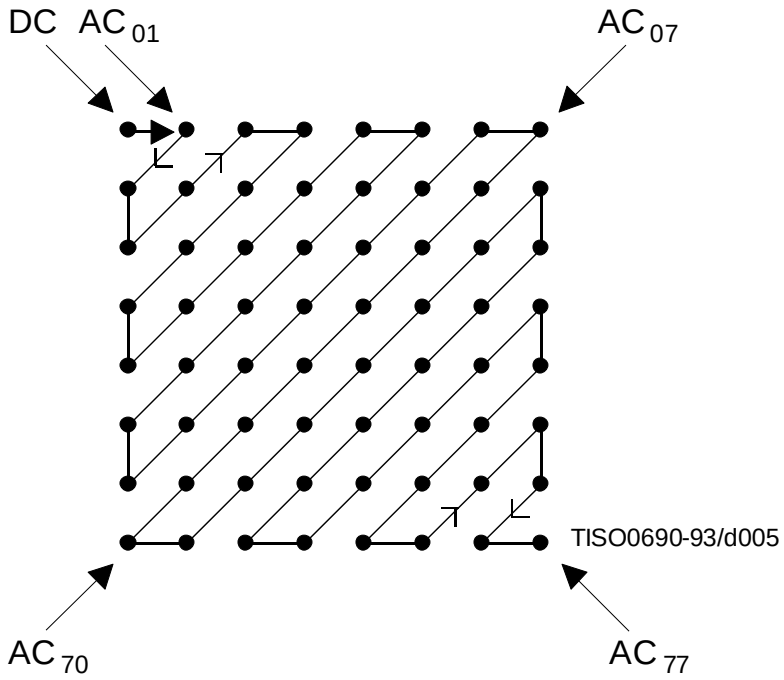
$$Q(u,v) = \frac{(QT(u,v) * SF + 50)}{100}$$

where QT(u,v) denotes the original quantization table coefficients, Q(u,v) denotes the adjusted quantization coefficients.

DC Encoding, AC Scanning



Differential DC encoding

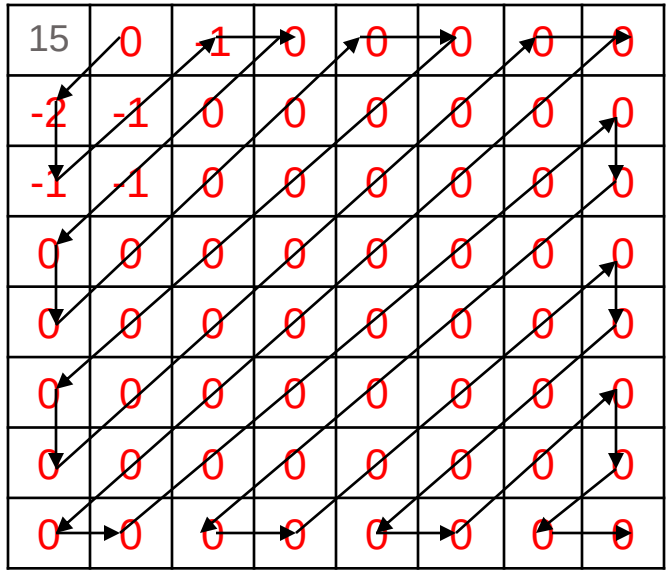


Zig-zag order

Figure 5 – Preparation of quantized coefficients for entropy encoding

AC Run-length encoding

- Run-length encoding



15 0 –2 –1 –1 –1 0 0 –1 0 0 …. 0

<1,-2> <0,-1> <0,-1> <0,-1> <2,-1> <EOB>

Differential Pulse Coded Modulation

- Since DC coefficients frequently contain a significant percentage of bitrate, special treatment is worthwhile.
- DPCM is adopted: using the previous QDC to predict the current QDC.

$$X_{DPCM} = X_i - X_{i-1}$$

- One predictor for each component.

DC=138	DC=140
DC=136	DC=138

DPCM ⟶ 138 => 2 => -4 => 2

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

DC Huffman Encoding

SSSS	DIFF values
0	0
1	-1,1
2	-3,-2,2,3
3	-7..-4,4..7
4	-15..-8,8..15
5	-13..-16,16..31
….	….

DC Diff magnitude category

Category	Code length	Code word
0	2	00
1	3	010
2	3	011
3	3	100
4	3	101
5	3	110
….	….	….

DC Huffman Table

DIFF values	Code word
-1,1	0,1
-3,-2,2,3	00,01,10,11

DC code word =

Category code word + DIFF value code word

15 → 101 1111

AC Huffman Encoding

SSSS	AC coefficients
1	-1,1
2	-3,-2,2,3
3	-7..-4,4..7
4	-15..-8,8,15
5	-13..-16,16..31
6	-127..-
….	64,64..127
	….

AC coefficient magnitude category

Run/Size	Code length	Code word
0/0 (EOB)	4	1010
0/1	2	00
..	..	..
1/2	5	11011
..	..	..
2/1	5	11100
….	….	….

AC Huffman Table

AC code word = Run/Size code word + AC coefficient code word

<1,-2> <0,-1> <0,-1> <0,-1> <2,-1> <EOB>

11011 01 00 0 00 0 00 0 11100 0 1010

Jing-Ming Guo (jmguo@seed.net.tw), Dept. EE, NTUST, Multimedia Signal Processing Lab.

Original

C.R.=4

C.R.=5.8

C.R.=9.5

Original

C.R.=7.7

C.R.=13.6

C.R.=23.2

References

[1] K. Sayood. *Introduction to Data Compression*, *3rd*. Elsevier Science Ltd., 2005

[2] Y. Q. Shi and H. Sun. *Image and Video Compression for Multimedia Engineering: Fundamentals, Algorighms, and Standards*. CRC Press, 2008.

[3] A. Gersho and R. M. Gray. *Vector Quantization and Signal Compression*. Springer, 1992.

[4] C. E. Shannon. *A Mathematical Theory of Communication*. Bell System Technical Journal, 27: 379-423, 623-656, 1948

[5] T. Berger. *Rate Distortion Theory: A Mathematical Basis for Data Compression*. Prentice-Hall, Englewood Cliffs, NJ, 1971

Deep learning Based Image Compression

[1] Balle, J., Laparra, V., & Simoncelli, E. P. (2017). *End-to-End Optimized Image* Compression, arXiv:1705.05823v1

=>This paper proposes an end-to-end image compression method based on autoencoders, achieving efficient compression by learning nonlinear transformations, making it particularly suitable for lossy compression.

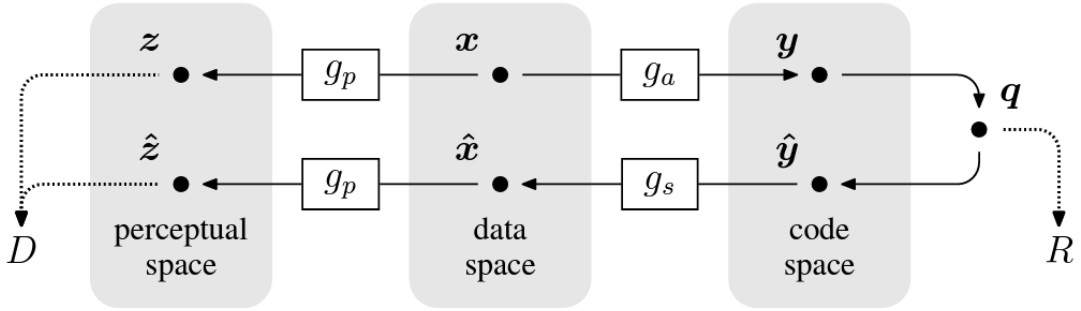


Figure 1: General nonlinear transform coding framework (Ballé, Laparra, and Simoncelli, 2016). A vector of image intensities $\boldsymbol{x} \in \mathbb{R}^N$ is mapped to a latent *code space* via a parametric *analysis* transform, $\boldsymbol{y} = g_a(\boldsymbol{x}; \boldsymbol{\phi})$. This representation is quantized, yielding a discrete-valued vector $\boldsymbol{q} \in \mathbb{Z}^M$ which is then compressed. The *rate* of this discrete code, R, is lower-bounded by the entropy of the discrete probability distribution of the quantized vector, $H[P_{\boldsymbol{q}}]$. To reconstruct the compressed image, the discrete elements of $\boldsymbol{q}$ are reinterpreted as a continuous-valued vector $\hat{\boldsymbol{y}}$, which is transformed back to the data space using a parametric *synthesis* transform $\hat{\boldsymbol{x}} = g_s(\hat{\boldsymbol{y}}; \boldsymbol{\theta})$. Distortion is assessed by transforming to a *perceptual space* using a (fixed) transform, $\hat{\boldsymbol{z}} = g_p(\hat{\boldsymbol{x}})$, and evaluating a metric $d(\boldsymbol{z}, \hat{\boldsymbol{z}})$. We optimize the parameter vectors $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$ for a weighted sum of the rate and distortion measures, $R + \lambda D$, over a set of images.

JPEG, 4283 bytes (0.121 bit/px), PSNR: luma 24.85 dB/chroma 29.23 dB, MS-SSIM: 0.8079

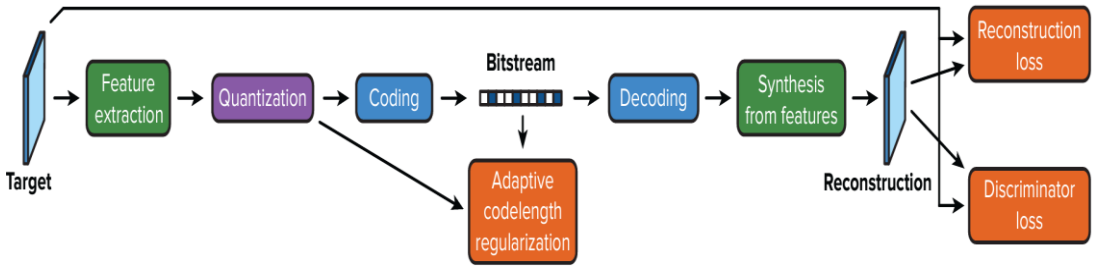
Proposed method, 3986 bytes (0.113 bit/px), PSNR: luma 27.01 dB/chroma 34.16 dB, MS-SSIM: 0.9039

JPEG 2000, 4004 bytes (0.113 bit/px), PSNR: luma 26.61 dB/chroma 33.88 dB, MS-SSIM: 0.8860

Deep learning Based Image Compression

[2] Rippel, O., & Bourdev, L. (2017). *Real-Time Adaptive Image Compression*, arXiv:1801.04260v4

=>This paper leverages Generative Adversarial Networks (GANs) for image compression, emphasizing the ability to maintain visual quality at low bitrates, making it an effective GAN-based compression method.



Deep Learning Based Image Compression

[3] Kingma, D. P., & Welling, M. (2014). *Auto-Encoding Variational Bayes*, arXiv:1703.10114v1

[4] Johnston, N., Vincent, D., & Mané, D. (2018). *Improved Lossy Image Compression with Priming and Spatially Adaptive Bit Rates for Recurrent Networks*, arXiv:1703.10114v1

[5] Ballé, J., Laparra, V., & Simoncelli, E. P. (2017). End-to-End Optimized Image Compression. *arXiv:1611.01704v4*

[6] Toderici, G., Vincent, D., Johnston, N., Hwang, S. J., Choi, D., & Viegas, F. (2017). Full Resolution Image Compression with Recurrent Neural Networks. *arXiv:1608.05148v2*

[7] Agustsson, E., Mentzer, F., Tschannen, M., Timofte, R., & Van Gool, L. (2018). Generative Adversarial Networks for Extreme Learned Image Compression. *arXiv:1804.02958v2*